





Université Cadi Ayyad

Faculté des sciences Semlalia Marrakech

Master en Intelligence Artificielle

Mémoire de Projet de Fin d'Études

Réalisé par :

Assia Bouamir

Mémoire présenté pour l'obtention du :

Diplôme de Master en Intelligence Artificielle

RAG Avancé pour l'Expansion Sémantique des Documents Juridiques

---

Entreprise d'accueil :
Institut de Recherche pour le Développement (IRD)

Date de soutenance : **30/06/2026**

ENCADRANTS

ENCADRANTE FSSM
Dr. Jihad Zahir

ENCADRANTE IRD
Dr. Marie Bonnin

MEMBRES DU JURY

EXAMINATRICE
Dr. Jihad Zahir

EXAMINATRICE
Dr. Latifa Errajy

EXAMINATRICE
Dr. Hasna El Alaoui

Année universitaire 2025-2026

Dédicace

À la petite fille qui se réveillait à cinq heures du matin pour prendre la route vers l'école,
À la petite fille qui a résisté aux difficultés et qui a fait l'impossible pour rendre son rêve possible,
À moi-même, pour ma persévérance, mon courage et ma détermination.

À mes chers parents,
Qui m'ont toujours laissée choisir mon propre chemin et poursuivre mes rêves sans m'imposer ceux des autres.
Merci pour votre amour, vos sacrifices et votre soutien inconditionnel.
Sans vous, je ne serais jamais arrivée jusque-là.

À mes encadrantes,
***Dr. Jihad Zahir** et **Dr. Marie Bonnin**,*
Pour leur accompagnement, leurs précieux conseils, leur disponibilité et leur confiance tout au long de ce travail.

*À mon professeur de licence, **Dr. Said El Abdlaoui**, ainsi qu'à toute l'équipe de la cellule d'écoute de l'École Supérieure de Technologie,*
Pour leur soutien, leurs encouragements, leur écoute attentive et leur bienveillance, qui ont été une source de motivation et de confiance tout au long de mon parcours.

Enfin,
À toutes les filles de la campagne,
Celles qui se battent chaque jour pour leurs études, leurs rêves et leur avenir.
Que ce mémoire soit la preuve que, malgré les obstacles, aucun rêve n'est trop grand lorsqu'on croit en soi et qu'on persévère.

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude et mes sincères remerciements à mon encadrante, **Dr. Jihad Zahir**, pour la confiance qu'elle m'a accordée dès le début de ce projet. Ses précieux conseils, sa rigueur scientifique, sa grande disponibilité et son soutien constant tout au long de ce travail ont été de formidables moteurs pour mener à bien ce mémoire.

Mes remerciements s'adressent également à ma co-encadrante, **Dr. Marie Bonnin**, pour son accompagnement, ses orientations enrichissantes et l'intérêt qu'elle a porté à mes recherches au sein de l'institut.

Je salue respectueusement les membres du jury, **Dr. Latifa Errajy** et **Dr. Hasna ElAlaoui**, qui ont accepté d'évaluer ce travail et de m'honorer de leur présence lors de cette soutenance.

Enfin, j'exprime ma reconnaissance envers l'ensemble du personnel de la Faculté des Sciences Semlalia de Marrakech (FSSM), de l'Institut de Recherche pour le Développement (IRD), pour m'avoir offert un environnement de travail stimulant et les ressources nécessaires à la concrétisation de ce projet.

Abstract

The exponential growth of legal documents and the complexity of their cross-references pose significant challenges for traditional Retrieval-Augmented Generation (RAG) systems. This thesis presents an advanced RAG architecture specialized in the semantic expansion of legal documents. The proposed system automatically resolves cross-references within and between texts, enriches each legal article with its full contextual dependencies, and stores semantically complete representations in a vector database.

The work is structured around two complementary axes : (1) a multi-graph and multi-agent RAG architecture that integrates knowledge graphs, hybrid retrieval (dense + sparse + graph-based), and intelligent orchestration to improve contextual fidelity and reduce hallucinations ; and (2) an autonomous dynamic ingestion pipeline capable of real-time monitoring of legislative sources, conflict detection, temporal deduplication, and automatic updating of the knowledge base.

Evaluated on a corpus of 50 legal documents (7,943 clauses) covering 17 Francophone jurisdictions, primarily in the field of marine environmental law, the system demonstrates substantial improvements over baseline Naive RAG approaches across RAGAS metrics, BERTScore, citation accuracy, and human evaluation.

This research contributes to the development of reliable, traceable, and up-to-date legal AI systems, particularly suited for low-resource and multilingual contexts.

Table des matières

Table des figures **7**

Liste des tableaux **9**

List of Acronyms **10**

Introduction **12**
Contexte général et problématique 12
Objectifs du stage 14
Plan du mémoire 16

1 État de l'art **17**
1.1 Introduction au Retrieval-Augmented Generation (RAG) 17
1.2 Architectures RAG : évolution des paradigmes 17
1.3 Gestion des références croisées dans les documents juridiques 18
1.4 Graphes de connaissances dans les systèmes RAG 19
1.5 Modèles adaptés au français et au domaine juridique 20
1.6 Évaluation des systèmes RAG 21
1.7 Travaux existants en traitement automatique des langues juridiques 21
1.8 Synthèse et positionnement de notre travail 22

2 Cadre méthodologique et outils **24**
2.1 Présentation de l'environnement de stage 24
2.2 Description du corpus juridique 25
2.2.1 Composition thématique et géographique 25
2.2.2 Statistiques détaillées par juridiction 25
2.2.3 Défis et caractéristiques spécifiques du corpus 26
2.3 Outils et technologies adoptés 27
2.3.1 Parsing et extraction : LlamaParse 27
2.3.2 Orchestration et retrieval : LangChain / LlamaIndex / LangGraph 28

2.3.3 Stockage vectoriel : ChromaDB / FAISS 29
2.3.4 Modèles LLM et embeddings 30
2.3.5 BM25 et graph retrieval 31
2.3.6 Contraintes et justification des choix technologiques 32
2.3.7 Architecture globale : articulation des deux axes 34

3 Conception et Implémentation **36**
3.1 Introduction 36
3.2 Axe 1 : Architecture RAG Multi-Graphes et Multi-Agents 37
3.2.1 Architecture globale 37
3.3 Parsing et préparation des documents juridiques 37
3.3.1 Parsing et Préparation des Documents Juridiques 38
3.3.2 Couche de Graphes de Connaissances 41
3.3.3 Moteur de Recherche Hybride 44
3.3.4 Orchestration Multi-Agents avec LangGraph 45
3.3.5 Diagrammes UML du système multi-agents 48
3.3.6 Standard RAG (Baseline) 49
3.4 Axe 2 : Pipeline d'Ingestion Dynamique Autonome 49
3.4.1 Architecture du pipeline d'ingestion autonome 49
3.4.2 PHASE A : Ingestion et Structuration (hors-ligne) 50
3.4.3 Gestion de l'évolution et de la cohérence temporelle 50
3.4.4 Veille juridique automatique et mise à jour continue 51
3.4.5 PHASE B : Recherche Hybride et Orchestration Multi-Agent (en-ligne) 52
3.4.6 Synthèse et intégration des deux axes 52

4 Expérimentations et Résultats **53**
4.1 Protocole Expérimental 53
4.1.1 Corpus et Benchmark 53
4.1.2 Métriques d'Évaluation 54
4.2 Résultats de l'Architecture RAG Multi-Graphes et Multi-Agents . . 55
4.2.1 Métriques RAGAS 55
4.2.2 BERTScore et Étude d'Ablation 56
4.2.3 Exactitude des Citations 57
4.2.4 Exemples Qualitatifs Contrastés 57
4.3 Résultats du Pipeline d'Ingestion Dynamique 58
4.3.1 Performance End-to-End 58
4.3.2 Résilience et Robustesse 59
4.3.3 Efficacité de la Veille Juridique et de la Déduplication 59
4.4 Discussion 59
4.4.1 Apports Principaux 59

4.4.2 Limites et Axes d'Amélioration 60

5 Conclusion Générale **61**
5.1 Rappel de la problématique et des objectifs 61
5.2 Synthèse des apports 62
5.3 Réponse à la question de recherche 62
5.4 Limites du travail 62
5.5 Perspectives de recherche 63

A Prompts des agents et du pipeline d'ingestion **64**
A.1 Prompt de parsing LlamaParse 64
A.2 Prompts du système multi-agents 65
A.2.1 Agent Routeur (router_agent) 65
A.2.2 Agent de Réponse (answering_agent) 66
A.2.3 Super-Agents (scripts/super_agents.py) 66
A.3 Captures d'écran des interfaces du système 67

Bibliographie **70**

Table des figures

2.1 Architecture globale du système RAG avancé : quatre couches intégrées couvrant le parsing des documents, la couche de graphes de connaissances, le moteur de recherche hybride et l'orchestration multi-agents via LangGraph. 34

3.1 Flux de données du PDF brut vers la structure `DocumentNode`. LlamaParse assure un parsing structuré de haute qualité (84,9% sur ParseBench). La normalisation Unicode NFKD réduit les artefacts OCR, tandis que l'enrichissement sémantique ajoute automatiquement la juridiction et la loi parente. Le résultat final est un `DocumentNode` normalisé et enrichi, prêt pour l'indexation et la construction des graphes. 39
3.2 Schéma hiérarchique du modèle de données `Document`. 40
3.3 Graphe structurel hiérarchique (arête `contains`). Ce graphe modélise la structure interne des documents juridiques. Chaque nœud représente un `DocumentNode` (article, section ou chapitre), et les arêtes dirigées indiquent la relation de containment. 42
3.4 Graphe des citations (`references`). Les nœuds représentent les articles juridiques et les arêtes les références explicites entre eux. Le parcours récursif permet d'inclure systématiquement les clauses dépendantes dans le contexte de récupération avec un score de pertinence maximal. 43
3.5 Exemple de graphe des définitions (arête `means`). Chaque terme technique est relié à sa définition officielle et à l'article de référence correspondant. Ce mécanisme permet d'injecter automatiquement les définitions dans le contexte de récupération. 44

3.6 Orchestration multi-agents avec LangGraph. Le `Supervisor_agent` coordonne le processus itératif en activant successivement les agents spécialisés. Après chaque cycle, il évalue la complétude du dossier juridique. L'`Answering_agent` n'est activé qu'après certification de la complétude par le superviseur. Un plafond de 3 itérations empêche les boucles infinies. 47
3.7 Diagramme de séquence UML montrant les interactions entre les cinq agents spécialisés 48
3.8 Diagramme d'activité UML du processus itératif d'orchestration multi-agents avec limite de 3 itérations 48

A.1 Interface principale du site juridique 68
A.2 Assistant RAG Intelligent – Interface de question-réponse 68
A.3 Module de veille juridique automatique 69
A.4 Pipeline d'ingestion automatique des documents juridiques 69

Liste des tableaux

1.1 Comparaison des trois paradigmes RAG. 18
1.2 Synthèse des principaux travaux en NLP juridique. 22

2.1 Composition du corpus par domaine thématique 25
2.2 Distribution corpus par juridiction (50 documents, 7,943 clauses) . . 26
2.3 Justification du choix de LlamaParse 27
2.4 Composants d'orchestration et de retrieval 29
2.5 Configuration du stockage vectoriel 30
2.6 Configuration des modèles LLM et des embeddings 31
2.7 Composants BM25 et recherche par graphe 31
2.8 Arbitrages technologiques selon les contraintes du projet 33

3.1 Comparaison des parseurs PDF. LlamaParse Agentic atteint 84,9% sur ParseBench (2000 pages vérifiées). 38
3.2 Robustesse des poids de fusion des scores. Les intervalles de confiance se chevauchent, confirmant la stabilité du signal BM25-dominant. . 45

4.1 Statistiques du corpus par thème. Tous textes en français. 53
4.2 Distribution clauses par juridiction (50 documents uniques = 7,943 clauses). 54
4.3 Scores RAGAS par thème. 56
4.4 BERTScore F1 poolé sur les deux thématiques évaluées (n=263). . . 56
4.5 Étude d'ablation — Thème Whale Hunting (n=131). Les intervalles de confiance non chevauchants confirment la significativité statistique. 56
4.6 Exactitude des citations (métrique binaire). 57
4.7 Exemples contrastés de réponses générées. 58
4.8 Performance du pipeline d'ingestion sur un document de 75 pages. . 58

Liste des acronymes

- **ANN :** Approximate Nearest Neighbors.
- **API :** Application Programming Interface.
- **BERT :** Bidirectional Encoder Representations from Transformers.
- **BGE-M3 :** BAAI General Embedding – Multilingual, Multi-granularity, Multi-functionality.
- **BM25 :** Best Match 25 (algorithme de scoring lexical).
- **CLERC :** Collection for Legal Retrieval and Classification.
- **CSR :** Corporate Social Responsibility.
- **DMR :** Document Mismatch Retrieval.
- **DPR :** Dense Passage Retrieval.
- **FAISS :** Facebook AI Similarity Search.
- **FCFA :** Franc CFA (Communauté Financière Africaine).
- **GDPR :** General Data Protection Regulation.
- **HNSW :** Hierarchical Navigable Small World.
- **HTML :** HyperText Markup Language.
- **HTTP :** HyperText Transfer Protocol.
- **IA :** Intelligence Artificielle.
- **IC :** Intervalle de Confiance.
- **IRD :** Institut de Recherche pour le Développement.
- **ISO :** International Organization for Standardization.
- **IVF :** Inverted File Index.

- **JSON :** JavaScript Object Notation.
- **KG :** Knowledge Graph (Graphe de connaissances).
- **LKIF :** Legal Knowledge Interchange Format.
- **LLM :** Large Language Model.
- **MARPOL :** International Convention for the Prevention of Pollution from Ships.
- **MD5 :** Message Digest 5 (algorithme de hachage).
- **MLP :** Multi-Layer Perceptron.
- **MRR :** Mean Reciprocal Rank.
- **NDCG :** Normalized Discounted Cumulative Gain.
- **NFKD :** Normalization Form Compatibility Decomposition.
- **NLI :** Natural Language Inference.
- **NLP :** Natural Language Processing.
- **OCR :** Optical Character Recognition.
- **ONG :** Organisation Non Gouvernementale.
- **PDF :** Portable Document Format.
- **PQ :** Product Quantization.
- **QA :** Question Answering.
- **RAG :** Retrieval-Augmented Generation.
- **RAGAS :** Retrieval-Augmented Generation Assessment Suite.
- **RAPTOR :** Recursive Abstractive Processing for Tree-Organized Retrieval.
- **RSS :** Really Simple Syndication.
- **RTL :** Right-To-Left.
- **SAULM-7B :** Supervised and Aligned Unified Legal Model (7 milliards de paramètres).
- **SHA-256 :** Secure Hash Algorithm 256 bits.
- **TALN :** Traitement Automatique du Langage Naturel.
- **UML :** Unified Modeling Language.
- **USD :** United States Dollar.

Introduction

Contexte général et problématique

Dans un contexte de transformation numérique mondiale accélérée, le secteur juridique est confronté à une croissance exponentielle du volume de documents textuels. Les textes législatifs, réglementaires, jurisprudentiels et contractuels constituent aujourd'hui un corpus massif, souvent structuré de manière hiérarchique et interconnecté par un dense réseau de références croisées (articles, annexes, définitions techniques ou renvois à d'autres textes). Cette complexité structurelle rend l'information rarement linéaire et pose des défis majeurs pour son traitement automatisé [1, 2].

Au niveau international, la numérisation des archives judiciaires et législatives a permis un accès plus large à l'information juridique, mais elle a également amplifié les difficultés d'exploitation. Les systèmes d'intelligence artificielle, et plus particulièrement les grands modèles de langage (LLM), ont émergé comme des outils prometteurs pour assister les professionnels du droit dans la recherche, l'analyse et la synthèse de ces documents. Cependant, ces modèles souffrent intrinsèquement de limitations liées à leurs connaissances figées à leur date d'entraînement et à leur propension aux hallucinations [3].

C'est dans ce cadre que les architectures de type *Retrieval-Augmented Generation* (RAG), introduites initialement pour enrichir les LLM par une récupération externe de connaissances [4], se sont imposées comme une solution de référence. Néanmoins, les approches RAG classiques reposent sur un découpage séquentiel (*chunking*) des documents en fragments de taille fixe ou sémantique. Cette méthode, bien qu'efficace pour des textes simples, entraîne une perte critique de contexte lorsqu'elle est appliquée aux documents juridiques. Un article de loi renvoyant à un autre article (« cf. Article 4.2 »), à une annexe ou à une définition située en fin de document peut perdre toute autonomie sémantique dès lors que le fragment isolé est extrait de son environnement référentiel [1, 2]. Ce phénomène, que nous désignerons par la suite sous le terme de *Document Level Retrieval Mismatch*, constitue un verrou majeur pour la fiabilité des systèmes RAG juridiques.

La problématique centrale de ce stage de recherche réside donc dans la conception

d'un pipeline RAG avancé capable de résoudre ces références croisées, d'enrichir sémantiquement chaque article et de stocker une version contextuellement complète dans une base vectorielle. Sans cette expansion sémantique, les systèmes RAG restent inadaptés au domaine juridique, où la précision et la traçabilité des sources sont des exigences non négociables [5, 3].

Importance du traitement automatique des documents juridiques

Le traitement automatique des documents juridiques par les techniques de traitement du langage naturel (NLP) et d'intelligence artificielle représente aujourd'hui un enjeu majeur, tant sur le plan économique que sociétal. Les professionnels du droit consacrent traditionnellement une part importante de leur temps à la recherche, à la lecture et à l'analyse de documents volumineux et complexes. L'automatisation de ces tâches permet de réduire considérablement ces délais, d'améliorer la précision des analyses et de démocratiser l'accès au droit.

Dans le domaine juridique, les bénéfices sont multiples. D'une part, les systèmes automatisés facilitent la conformité réglementaire, la détection des risques contractuels et la prédiction des décisions judiciaires. D'autre part, ils contribuent à lutter contre la fracture numérique en offrant aux citoyens, aux entreprises et aux administrations une assistance juridique plus rapide et plus accessible.

De plus, le traitement automatique permet de préserver la traçabilité et la transparence des raisonnements juridiques. Contrairement aux modèles de langage purs, les approches hybrides (telles que les systèmes RAG enrichis par des graphes de connaissances) fournissent des sources explicites et vérifiables. Cette caractéristique répond directement aux exigences éthiques et déontologiques de la profession, notamment l'obligation de motivation des décisions et le devoir de conseil fiable.

Enfin, sur le plan scientifique, le développement de pipelines spécialisés pour les documents juridiques contribue à l'avancement des recherches en *NLP juridique*, particulièrement dans les contextes multilingues et faiblement dotés en ressources. Ce travail ouvre la voie à des applications concrètes telles que des assistants virtuels pour les magistrats, des outils de veille législative automatisée ou encore des systèmes d'aide à la rédaction normative.

Limites des approches RAG classiques

Les architectures RAG classiques, bien qu'elles constituent une avancée majeure par rapport aux LLM utilisés seuls, présentent plusieurs limitations structurelles qui les rendent particulièrement inadaptées au traitement des documents juridiques :

1. **Perte du contexte référentiel :** Le découpage séquentiel des textes en *chunks* (de taille fixe ou sémantique) entraîne une perte systématique du contexte. Un article de loi faisant référence à un autre passage (« cf. Article 4.2 »), à une annexe ou à une définition se retrouve isolé de ses liens, ce qui rend le fragment incomplet et sémantiquement dépendant [2, 1].
2. **Document-Level Retrieval Mismatch :** Cette fragmentation provoque des erreurs de ciblage, le module de récupération sélectionnant parfois des passages issus de sections inappropriées. Cela s'explique par la forte similarité structurelle et lexicale des textes juridiques, ce qui conduit le modèle à générer des réponses plausibles mais factuellement erronées, augmentant ainsi le taux d'hallucinations [1].
3. **Absence de gestion des relations complexes :** Les approches RAG classiques sont incapables de modéliser les relations hiérarchiques, temporelles ou croisées propres au domaine juridique (versions successives d'un texte, définitions globales, renvois inter-articles, etc.). L'absence de graphes de connaissances limite fortement la reconstruction du contexte global indispensable à une interprétation fiable [2].
4. **Inadéquation des métriques traditionnelles :** Les critères d'évaluation classiques (tels que le *Recall@K* ou la similarité cosinus) ne permettent pas de mesurer fidèlement la pertinence contextuelle ni la réduction des hallucinations, alors que la précision absolue et la traçabilité des sources sont pourtant essentielles dans le domaine juridique [5].
5. **Absence de détection des évolutions législatives :** Les systèmes RAG classiques ne disposent d'aucun mécanisme permettant d'identifier les mises à jour, amendements ou abrogations affectant les textes déjà indexés. Lorsqu'un nouveau texte modifie une disposition existante, la base de connaissances peut contenir simultanément deux versions contradictoires d'une même règle, ce qui dégrade la fiabilité des réponses générées et expose le système à des risques de désinformation juridique.

Ces limites expliquent pourquoi les systèmes RAG classiques restent insuffisants pour les applications juridiques critiques. Le présent travail propose d'y répondre à travers une architecture RAG avancée combinant modélisation des relations complexes et gestion dynamique des évolutions normatives.

Objectifs du stage

L'objectif principal de ce stage est de concevoir et d'implémenter un pipeline RAG avancé pour le traitement des documents juridiques, capable de résoudre automatiquement les références croisées et d'enrichir sémantiquement chaque article

afin de produire des représentations contextuellement complètes. Ce pipeline intègre également un mécanisme de détection des évolutions législatives lors de l'ingestion, afin d'assurer la cohérence et l'actualité de la base de connaissances.

Les objectifs spécifiques sont les suivants :

— Modéliser les références croisées juridiques à l'aide d'une architecture multi-graphe (graphe structurel, graphe des citations et graphe des définitions).

— Mettre en place une orchestration multi-agents permettant de garantir la complétude du contexte juridique avant la génération de réponses.

— Développer un pipeline d'ingestion autonome capable de détecter et d'intégrer les nouveaux documents législatifs en temps réel.

— Assurer la cohérence temporelle de la base de connaissances par déduplication et détection automatique des conflits entre versions.

— Évaluer le système de manière rigoureuse à l'aide de métriques adaptées au domaine juridique et d'une évaluation humaine.

Question de recherche et hypothèse principale

Question de recherche principale : Comment concevoir et implémenter un pipeline RAG avancé capable de résoudre automatiquement les références croisées dans les documents juridiques par une expansion sémantique intelligente, tout en détectant les évolutions législatives lors de l'ingestion, afin de produire des représentations contextuellement complètes et de réduire les hallucinations ?

Hypothèse principale : L'intégration d'un mécanisme hybride de résolution de références (règles linguistiques + LLM + graphe de connaissances), combinée à une expansion sémantique des articles et à un module de détection des évolutions normatives à l'ingestion, permettra d'obtenir une amélioration significative de la fidélité contextuelle (Faithfulness), de la précision du contexte et de la réduction des hallucinations, par rapport à une architecture RAG classique fondée sur un chunking séquentiel simple.

Cette hypothèse repose sur le postulat selon lequel la perte de contexte référentiel et l'obsolescence des données indexées constituent les deux principales sources d'erreur dans les systèmes RAG juridiques. Leur résolution explicite — via un graphe relationnel enrichi d'une part, et un pipeline d'ingestion dynamique sensible aux évolutions législatives d'autre part — est considérée comme nécessaire et suffisante pour atteindre un niveau de performance adapté aux exigences de précision du domaine juridique.

Plan du mémoire

Ce mémoire est structuré en cinq chapitres principaux, organisés de manière logique pour répondre à la question de recherche posée :

- ⋆ **Chapitre 1 – État de l'art** : Présente les architectures RAG existantes, les techniques de gestion des références croisées, les graphes de connaissances et les travaux en NLP juridique, afin de positionner précisément notre contribution.
- ⋆ **Chapitre 2 – Cadre méthodologique et outils** : Décrit l'environnement du stage, le corpus juridique, les choix technologiques (LangChain/LlamaIndex, ChromaDB, modèles open-source) et la méthodologie globale d'expansion sémantique et de gestion des évolutions normatives.
- ⋆ **Chapitre 3 – Conception et implémentation** : Détaille l'architecture proposée, l'algorithme d'expansion, la construction du graphe de connaissances, le pipeline RAG complet et le framework d'ingestion dynamique LegalStreamRAG, avec schémas UML et diagrammes de flux.
- ⋆ **Chapitre 4 – Expérimentations et résultats** : Présente le protocole expérimental, les métriques utilisées, les résultats quantitatifs et qualitatifs, ainsi que l'étude d'ablation et la discussion des limites.
- ⋆ **Conclusion générale** : Synthétise les apports, répond à la question de recherche, identifie les limites et propose des perspectives de recherche future.

Des annexes techniques (code source complet, prompts utilisés, jeux de données d'évaluation) et une bibliographie exhaustive complètent le document.

Chapitre 1

État de l'art

1.1 Introduction au Retrieval-Augmented Generation (RAG)

Le Retrieval-Augmented Generation (RAG) est une architecture hybride qui combine les capacités génératives des grands modèles de langage (LLM) avec un mécanisme de récupération d'information externe. Introduit formellement par Lewis et al. [4], le RAG a été conçu pour pallier les principales limites des LLM : connaissances figées à la date d'entraînement, propension aux hallucinations et difficulté à intégrer des informations récentes ou spécifiques à un domaine [5, 6].

Le fonctionnement du RAG repose sur trois étapes principales : (1) la **récupération** des passages les plus pertinents depuis une base de connaissances externe, (2) l'**enrichissement** de la requête avec ces passages, et (3) la **génération** d'une réponse conditionnée par ce contexte enrichi. Cette approche permet d'améliorer significativement la factualité et la traçabilité des réponses générées [4].

Depuis son introduction, le RAG s'est imposé comme l'une des méthodes les plus efficaces pour combiner recherche sémantique et génération conditionnée, notamment dans les applications nécessitant une haute fiabilité [5, 7]. Des revues récentes estiment que le RAG réduit le taux d'hallucinations de 30 à 50 % par rapport à une génération directe sans contexte externe [6].

1.2 Architectures RAG : évolution des paradigmes

Les architectures RAG ont connu une évolution rapide. On distingue généralement trois paradigmes principaux [5, 8] :

— **RAG de base (Naive RAG)** : correspond à l'implémentation la plus simple. Elle repose sur un index vectoriel statique et une génération directe à partir

des k passages les plus similaires. Bien qu'elle soit rapide à mettre en œuvre, cette approche souffre de plusieurs faiblesses : sensibilité au bruit dans les chunks récupérés, absence d'optimisation post-retrieval, et difficulté à gérer des requêtes complexes ou multi-sauts [5].

— **RAG avancé (Advanced RAG)** : introduit des optimisations avant et après la récupération. Les techniques pré-retrieval incluent la reformulation de requête (*query expansion*, HyDE) et le découpage hiérarchique ou sémantique. Les techniques post-retrieval comprennent le reranking par cross-encoder, la compression de contexte et la suppression de redondances [5]. Ces ajouts permettent d'améliorer la précision du contexte fourni au LLM et de réduire les hallucinations.

— **RAG modulaire (Modular RAG)** : représente le stade le plus mature. Il décompose le pipeline en modules interchangeables et supporte des flux dynamiques, l'intégration de graphes de connaissances ainsi que des mécanismes d'orchestration [8]. Des variantes notables incluent Self-RAG [9], qui entraîne le LLM à décider quand et quoi récupérer, et RAPTOR [10], qui construit une arborescence résumée du corpus pour une récupération hiérarchique. Ce paradigme constitue le cadre dans lequel s'inscrit notre travail.

Le tableau 1.1 résume les principales caractéristiques de ces trois paradigmes.

Table 1.1 – Comparaison des trois paradigmes RAG.

Caractéristique	**RAG de base**	**RAG avancé**	**RAG modulaire**
Complexité	Faible	Moyenne	Élevée
Optimisations de récupération	Aucune	Pré et post	Multiples + routage
Flexibilité	Faible	Moyenne	Très élevée
Performance sur cas complexes	Faible	Bonne	Excellente
Cas d'usage typique	Prototypes	Production	Systèmes avancés

1.3 Gestion des références croisées dans les documents juridiques

Les documents juridiques se caractérisent par une forte densité de relations explicites et implicites : renvois inter-articles, définitions, annexes et modifications successives. Le découpage séquentiel classique rompt ces liens : un article contenant une référence à un autre texte perd une grande partie de sa signification lorsqu'il est isolé de son contexte [1, 2].

Ce phénomène, que Reuter et al. [1] désignent sous le terme de *Document-Level Retrieval Mismatch* (DMR), constitue l'une des principales sources d'erreur dans les systèmes RAG juridiques. Empiriquement, les auteurs estiment que jusqu'à 40 % des fragments extraits par une approche de découpage naïf sont sémantiquement incomplets en raison de références non résolues.

Plusieurs stratégies ont été proposées pour atténuer ce problème :

— **Règles heuristiques et parsing structuré** : utilisation d'expressions régulières et de grammaires spécifiques au style juridique pour extraire les motifs de référence [11].

— **Résolution de coréférence et entity linking** : modèles tels que spaCy ou des LLMs en mode *few-shot* pour relier les pronoms et mentions (« le présent article », « ladite définition ») à leurs entités cibles.

— **Découpage contextuel (*parent-document retrieval*)** : conservation d'un contexte large lors de la génération, même si le retrieval porte sur un sous-fragment [5].

— **Retrieval par graphe (*graph-based retrieval*)** : modélisation explicite des relations sous forme de graphe dirigé, permettant un parcours récursif lors de la récupération [2, 12].

Les travaux les plus récents montrent que les méthodes hybrides (règles + apprentissage) surpassent les approches purement neuronales sur les corpus juridiques en raison de la régularité stylistique des textes de loi [1]. Cependant, la plupart des travaux existants restent limités à la jurisprudence et intègrent rarement une expansion sémantique systématique dans un pipeline RAG complet couvrant les textes normatifs.

1.4 Graphes de connaissances dans les systèmes RAG

Les graphes de connaissances (KG) constituent un outil particulièrement adapté à la représentation des relations complexes présentes dans le droit. Dans ce domaine, un KG juridique typique comporte : des nœuds représentant articles, définitions, lois et jurisprudences ; et des arêtes de type *définit*, *modifie*, *renvoieVers*, *estAbrogéPar* [2].

Plusieurs initiatives ont formalisé des KG juridiques : EuroVoc et ses extensions nationales, le Legal Knowledge Interchange Format (LKIF), et des projets récents utilisant des LLMs pour la construction automatique de graphes à partir de textes bruts. L'approche GraphRAG de Microsoft [12] constitue une avancée notable : en construisant un graphe de communautés à partir de résumés hiérarchiques, elle

surpasse le RAG vectoriel standard de 20 à 40 % sur les tâches de raisonnement multi-sauts.

Dans le cadre du RAG, l'intégration d'un KG offre plusieurs avantages : un retrieval guidé par le graphe plutôt que par similarité vectorielle pure, un contexte enrichi par des triplets sémantiques, et une meilleure gestion de la temporalité (versions successives, amendements, abrogations) [2].

Dans le domaine juridique spécifiquement, les approches basées sur des graphes ont démontré leur intérêt pour les tâches nécessitant une compréhension multi-sauts [2, 3]. Néanmoins, les travaux existants se concentrent majoritairement sur la jurisprudence plutôt que sur les textes normatifs. De plus, peu d'approches combinent une modélisation multi-graphe (structurel + citationnel + définitionnel) avec une orchestration intelligente et un pipeline d'ingestion dynamique. C'est précisément dans cette direction que se positionne notre contribution.

1.5 Modèles adaptés au français et au domaine juridique

Le français juridique présente des défis spécifiques liés au vocabulaire technique très spécialisé, aux structures syntaxiques longues et formelles, et à la fréquence élevée de termes latins. Ces caractéristiques rendent les modèles généralistes sous-optimaux [3].

Modèles d'embeddings adaptés :

— **CamemBERT, FlauBERT** : modèles BERT francophones pré-entraînés sur des corpus généralistes (OSCAR, CCNet) [13, 14].

— **CamemBERTavocat, JuriBERT** : versions fine-tunées spécifiquement sur des corpus juridiques francophones (décisions judiciaires, codes, doctrine). JuriBERT [15] atteint des gains de +5 à +8 % de F1 par rapport à CamemBERT sur les tâches d'extraction d'entités juridiques. CamemBERTavocat [16] est optimisé pour la classification de décisions de justice françaises.

— **BGE-M3** : modèle multilingue récent (dense + sparse) montrant de bonnes performances zéro-shot sur le français juridique [17].

LLMs adaptés au français et/ou au juridique :

— **Mistral-7B, Mixtral-8x7B** : excellentes performances en français, souvent supérieures à LLaMA-3 sur des tâches francophones [18].

— **SAULM-7B, Legal-Mistral** : adaptations de Mistral sur des corpus juridiques multilingues, avec un accent sur le raisonnement en chaîne et la génération fidèle [19].

Les évaluations récentes indiquent que combiner un embedding juridique francophone (CamemBERTavocat ou BGE-M3) avec un LLM comme Mistral-7B-Instruct offre actuellement le meilleur compromis performance/coût/accessibilité pour les applications RAG en droit français [3].

1.6 Évaluation des systèmes RAG

L'évaluation des systèmes RAG est particulièrement délicate car elle doit mesurer simultanément la qualité de la récupération, la pertinence du contexte et la fidélité de la génération [20].

Métriques de récupération : Recall@K, Precision@K, Mean Reciprocal Rank (MRR) et Normalized Discounted Cumulative Gain (NDCG@K).

Métriques de cohérence du contexte :

— *Context Precision* : proportion des passages récupérés réellement pertinents pour la question ;

— *Context Recall* : proportion des informations nécessaires effectivement présentes dans les passages récupérés.

Métriques de génération (end-to-end) :

— *Faithfulness* : proportion des affirmations de la réponse directement supportées par les passages récupérés ;

— *Answer Relevancy* : similarité sémantique entre la réponse et la question ;

— *Answer Correctness* : combinaison de similarité factuelle et sémantique par rapport à une référence *gold* ;

— *RAGAS score* : score composite agrégeant les quatre métriques précédentes [20].

Dans le domaine juridique, les métriques automatiques montrent des corrélations modérées avec les jugements humains sur les tâches complexes (raisonnement multi-sauts, interprétation contextuelle), ce qui justifie une évaluation hybride systématique [3]. Les métriques humaines (échelle de Likert, annotation binaire des hallucinations, accord inter-annotateurs κ) restent indispensables en raison des enjeux de précision et de responsabilité propres au domaine juridique.

1.7 Travaux existants en traitement automatique des langues juridiques

Le traitement automatique du langage juridique a connu des avancées importantes depuis l'émergence des transformers. Le tableau 1.2 synthétise les principaux travaux de référence.

Table 1.2 – Synthèse des principaux travaux en NLP juridique.

Modèle / Système	Année	Langue	Tâche principale	Réf.
Legal-BERT	2020	EN	Classification, NER juridique	[11]
JuriBERT	2021	FR	NER, classification décisions	[15]
CamemBERTavocat	2023	FR	Classification décisions de justice	[16]
SAUL / SAULM-7B	2024	EN/FR	QA juridique, génération fidèle	[19]
Graph RAG (norms)	2025	FR	Retrieval normatif multi-sauts	[2]
Reliable RAG	2025	EN/FR	RAG sur grands corpus légaux	[1]

Legal-BERT [11] a été l'un des premiers modèles BERT fine-tunés sur un large corpus juridique anglophone (contrats, jurisprudence américaine, codes européens). Il a servi de référence pendant plusieurs années et reste une baseline solide.

SAUL / SAULM-7B [19] représente une adaptation récente de Mistral-7B sur des corpus juridiques multilingues, avec un accent sur le raisonnement en chaîne et la génération fidèle. Le modèle surpasse GPT-3.5 sur plusieurs benchmarks juridiques.

Concernant les approches RAG appliquées au droit :

— De Martim [2] propose d'utiliser des graphes hiérarchiques et temporels pour modéliser les évolutions législatives et les renvois normatifs. Cette approche améliore le Context Recall de +18 % par rapport à un RAG vectoriel standard sur un corpus de normes françaises.

— Reuter et al. [1] analysent les causes du Document-Level Retrieval Mismatch et proposent des stratégies de récupération résilientes pour les grands corpus juridiques.

Globalement, la majorité des systèmes existants se concentrent sur la jurisprudence plutôt que sur les textes normatifs (codes, lois, règlements), et très peu intègrent une expansion sémantique systématique des références croisées combinée à une indexation vectorielle enrichie et une gestion dynamique des évolutions législatives.

1.8 Synthèse et positionnement de notre travail

L'état de l'art révèle plusieurs lacunes persistantes dans le traitement des textes normatifs :

— Les approches classiques de découpage et de récupération ne préservent pas les relations référentielles essentielles au domaine juridique. Le phénomène de DMR peut affecter jusqu'à 40 % des fragments extraits [1].

— Les systèmes basés sur des graphes restent relativement marginaux et rarement intégrés dans des pipelines RAG complets et reproductibles. GraphRAG [12] constitue une exception notable mais se concentre sur des corpus généralistes en anglais.

— La gestion dynamique des évolutions législatives (détection des amendements, abrogations et versions successives) est peu ou pas traitée dans les systèmes RAG existants.

— Les travaux en français et dans les contextes multilingues à faibles ressources restent sous-représentés par rapport à l'anglais [3].

Notre travail se positionne comme une contribution visant à combler ces lacunes à travers **deux axes complémentaires** :

1. Une **architecture RAG multi-graphe et multi-agent** combinant un graphe structurel, un graphe des citations et un graphe des définitions, avec une orchestration LangGraph permettant de garantir la complétude du contexte juridique avant génération. Contrairement à GraphRAG [12], notre approche est spécifiquement conçue pour les textes normatifs francophones et intègre trois graphes complémentaires plutôt qu'un graphe unique de communautés.
2. Un **pipeline d'ingestion dynamique autonome** capable de détecter les évolutions législatives, d'effectuer une déduplication temporelle et de maintenir la cohérence de la base de connaissances au fil du temps. Ce composant est absent de la quasi-totalité des systèmes RAG évalués dans la littérature [5, 1].

Cette approche hybride (règles linguistiques + modèle de langage + graphe de connaissances) vise à offrir un bon compromis entre performance, interprétabilité et fiabilité juridique, tout en restant déployable dans des contextes à faibles ressources computationnelles.

Chapitre 2

Cadre méthodologique et outils

2.1 Présentation de l'environnement de stage

Ce stage s'inscrit dans une collaboration entre la Faculté des Sciences Semlalia (Université Cadi Ayyad, Marrakech) et l'Institut de Recherche pour le Développement (IRD).

L'IRD est un établissement public à caractère scientifique et technologique (EPST) français, placé sous la double tutelle du ministère chargé de l'Enseignement supérieur, de la Recherche et de l'Espace, et du ministère chargé des Affaires étrangères [1]. Créé en 1943 et dont le siège est établi à Marseille depuis 2008, l'IRD est présent dans plus de 50 pays d'Afrique, d'Amérique latine, d'Asie et du Pacifique. Il s'appuie sur un réseau de 34 représentations à l'étranger, 80 unités mixtes de recherche et 38 laboratoires mixtes internationaux, et regroupe plus de 2 300 agents dont 946 chercheurs. En 2023, 64 % de ses publications scientifiques ont été co-produites avec un partenaire du Sud.

Sa mission est de déployer une recherche fondée sur un partenariat scientifique équitable avec les pays du Sud, principalement ceux des zones intertropicales et méditerranéennes, au service des Objectifs de Développement Durable (ODD) des Nations Unies. L'IRD intervient selon trois axes complémentaires : la recherche partenariale, l'expertise et le conseil, et le renforcement des capacités scientifiques dans les pays du Sud.

Dans le cadre de ce stage, la collaboration avec l'IRD s'inscrit dans son axe de recherche sur la gouvernance marine et les régulations environnementales en Afrique francophone. L'institut a apporté un accès à des collections de documents juridiques spécialisés, une expertise pluridisciplinaire associant sciences marines et droit de l'environnement, ainsi qu'un réseau de juristes spécialisés en droit francophone africain.

1. `https://www.ird.fr/lird-presentation-et-historique`

Le projet bénéficie ainsi d'un cadre interdisciplinaire associant des compétences en intelligence artificielle (Université Cadi Ayyad) et en droit de l'environnement (IRD). Des échanges réguliers entre chercheurs en IA et experts juridiques ont permis une meilleure prise en compte des spécificités du domaine juridique dans le développement des méthodes proposées : définition des cas d'usage, constitution du corpus et validation du benchmark d'évaluation par des juristes spécialisés en droit africain francophone.

2.2 Description du corpus juridique

2.2.1 Composition thématique et géographique

Le corpus juridique utilisé dans cette étude rassemble 50 documents PDF uniques couvrant 17 juridictions distinctes et quatre domaines thématiques majeurs. Le tableau 4.1 synthétise cette composition.

Domaine thématique	**Documents**	**Clauses**	**Juridictions**
Chasse baleinière et protection des cétacés	23	2 080	12
Protection des oiseaux marins	14	957	8
Régulation des rejets d'hydrocarbures (MARPOL)	21	4 906	15
Total	**50**	**7 943**	**17**

TABLE 2.1 – Composition du corpus par domaine thématique

Composition géographique (par juridiction) : Le corpus couvre les juridictions africaines francophones suivantes : Algérie, Bénin, Cameroun, Comores, Congo, Côte d'Ivoire, Djibouti, Gabon, Guinée, Madagascar, Mauritanie, Maroc, Sénégal, Togo, Tunisie ainsi que la France et les instruments internationaux (convention MARPOL). La structure finale comprend 7,943 clauses qualifiées post-normalisation, dénommées `DocumentNodes`.

Ces textes révèlent une diversité remarquable : des lois promulguées entre 1975 et 2024, des codes régionalisés couverts par une législation commune (notamment le Droit commercial dans la Zone de Franc-Afrique), et un enchevêtrement de normes internationales (conventions MARPOL, droit de la mer) avec les implémentations nationales.

2.2.2 Statistiques détaillées par juridiction

Le tableau 2.2 détaille la distribution des documents et clauses par juridiction.

Juridiction	Documents	Clauses
Algérie	4	621
Bénin	3	418
Cameroun	4	593
Comores	2	284
Congo	2	310
Côte d'Ivoire	3	452
Djibouti	2	195
France	3	489
Gabon	4	601
Guinée	2	263
Madagascar	3	374
MARPOL (International)	3	712
Mauritanie	2	297
Maroc	4	538
Sénégal	4	612
Togo	2	287
Tunisie	3	397
Total	**50**	**7,943**

TABLE 2.2 – Distribution corpus par juridiction (50 documents, 7,943 clauses)

2.2.3 Défis et caractéristiques spécifiques du corpus

Hétérogénéité des formats

Le corpus présente une diversité substantielle de formats documents :

— **Documents numériques natifs** : Décrets et lois publiés par les administrations centrales (formats PDF).

— **Documents scannés** : Circulaires, ordonnances, textes d'archives numérisés par OCR.

— **Layouts complexes** : Textes multi-colonnes (particulièrement codes environnementaux du Cameroun et Madagascar), avec en-têtes, numérotation marginale non-standard.

— **Polices et encodages** : Mélange de caractères latins, accents diacritiques français, et (dans les versions préparées) caractères arabes bidirectionnels.

Caractéristiques linguistiques

— **Langue principale** : français (corpus complet évalué nativement sans traduction).

— **Variantes dialectales** : terminologie juridique francophone africaine distincte de la norme métropolitaine (p. ex. *circularisation* en droit camerounais vs. *notification* métropolitain).

— **Multilinguisme structurel** : textes fondamentaux rédigés en français, interprétations judiciaires pouvant incorporer terminologie locale.

Défis spécifiques à la structuration juridique

— **Références implicites** : De nombreuses clauses contiennent des références croisées non-explicites, supposant la connaissance du contexte législatif.

— **Versioning non-déclaré** : Modifications de lois sans versioning explicite (p. ex., Loi 82-10 du 26 août 1982 en Algérie vs. Loi 04-07 de 2004).

— **Hiérarchie article–section–chapitre** : La structure juridique impose une organisation hiérarchique multi-niveaux qui doit être préservée.

2.3 Outils et technologies adoptés

2.3.1 Parsing et extraction : LlamaParse

Le parsing des documents PDF revêt une importance critique pour le maintien de l'intégrité structurelle dans les pipelines de **Retrieval-Augmented Generation** (RAG) [4]. Nous employons **LlamaParse** (LlamaIndex) pour trois raisons principales :

Aspect	**Détail**
Outil	LlamaParse (LlamaIndex)
Fonction	Parsing structuré PDF → JSON hiérarchisé
Performance	8.68 s/doc (34% du coût total d'ingestion)
Format de sortie	JSON : article → section → chapitre
Justification	Seul candidat maintenant la fidélité structurelle et la validation du schéma sur un corpus hétérogène

TABLE 2.3 – Justification du choix de LlamaParse

Parmi les alternatives évaluées (reducto, doocling, grok), LlamaParse s'est distingué par sa capacité à préserver la hiérarchie logique des articles et à maintenir l'intégrité du schéma JSON même sur des documents multi-colonnes et scannés.

Évaluation indépendante : ParseBench (2026)

Cette évaluation interne est corroborée par le benchmark indépendant **ParseBench** [21], publié par l'équipe LlamaIndex et consacré à l'évaluation de la qualité du parsing documentaire pour les agents IA. ParseBench évalue 14 méthodes (modèles vision-langage généralistes, parseurs spécialisés tels que Docling et Reducto, ainsi que LlamaParse) sur environ 2 000 pages de documents d'entreprise vérifiées manuellement, selon cinq dimensions : tableaux, graphiques, fidélité du contenu, formatage sémantique et ancrage visuel.

Sur ce benchmark, **LlamaParse en mode Agentic** se classe parmi les meilleurs systèmes, et se distingue particulièrement sur les dimensions directement pertinentes pour notre corpus juridique et financier structuré [1, 3] :

— **Graphiques** : LlamaParse Agentic obtient le meilleur score avec **78,1%**. C'est une performance notable dans une dimension où la majorité des parseurs spécialisés restent sous les 6%, faute d'extraction structurée des données graphiques.

— **Ancrage visuel** (traçabilité du contenu extrait vers sa position source dans le document original) : LlamaParse Agentic mène le classement avec **80,6%**, devant Azure Document Intelligence (73,8%) et Amazon Textract (70,4%).

— **Fidélité du contenu** : LlamaParse figure parmi les meilleurs systèmes, bien que les auteurs soulignent que même les outils les plus performants n'atteignent qu'environ 90% sur cette dimension. Ce constat justifie l'ajout d'étapes de validation de schéma et d'auto-réparation JSON en aval du parsing (section 3.3.4).

Ces résultats confirment la pertinence de LlamaParse pour notre pipeline RAG appliqué au domaine juridique, notamment sa capacité à préserver à la fois la structure hiérarchique et les éléments visuels complexes des documents.

2.3.2 Orchestration et retrieval : LangChain / LlamaIndex / LangGraph

L'orchestration du système RAG repose sur une architecture multi-agents contrôlée, implémentée avec **LangGraph**. Contrairement aux systèmes multi-agents probabilistes, nous adoptons un **StateGraph** déterministe qui garantit un contrôle strict du raisonnement, une traçabilité complète des décisions et une limitation explicite des itérations — des propriétés essentielles dans le domaine juridique.

Le cœur de cette orchestration est assuré par un **StateGraph** composé de cinq agents spécialisés, coordonnés par un `Supervisor_agent` central :

Composant	Outil	Fonction
Retrieval orchestration	LangChain	Composition des chaînes de récupération hybrides
Document indexing	LlamaIndex	Indexation des `DocumentNode` enrichis
Multi-agent workflow	LangGraph StateGraph	Machine à états avec cinq agents spécialisés
Scheduling	APScheduler	Veille juridique autonome (intervalle de 6 heures)

TABLE 2.4 – Composants d'orchestration et de retrieval

— `Supervisor_agent` : Évalue l'état du dossier juridique après chaque étape et décide du prochain agent à activer. Il impose un plafond strict de trois itérations afin d'éviter les boucles infinies.

— `Definition_agent` : Interroge le graphe des définitions pour injecter les définitions légales officielles des termes non résolus.

— `Reference_detector_agent` : Identifie les références croisées explicites dans les clauses récupérées.

— `Cross_reference_resolution_agent` : Parcours le graphe des citations pour résoudre les références en attente et promeut les clauses dépendantes à un score de pertinence maximal.

— `Answering_agent` : Génère la réponse finale uniquement lorsque le Supervisor a certifié que le dossier juridique est complet.

Ce workflow itératif et stateful permet d'appliquer une **discipline de citation** stricte : aucune réponse n'est produite tant que toutes les références et définitions nécessaires n'ont pas été résolues. Cette approche constitue l'un des apports principaux de notre architecture par rapport aux systèmes RAG classiques à passage unique.

2.3.3 Stockage vectoriel : ChromaDB / FAISS

Le stockage vectoriel joue un rôle central dans l'architecture, car il doit permettre à la fois une récupération hybride efficace et un filtrage précis par métadonnées juridiques. Nous adoptons une stratégie duale : **ChromaDB** comme solution principale en phase de développement et d'expérimentation, et **FAISS** comme alternative de passage à l'échelle lorsque le corpus dépasse une certaine taille.

ChromaDB est retenu comme base principale pendant les phases de développement et d'évaluation. Il offre une API simple, une persistance native et une

Composant	Détail
Base de données embedding	ChromaDB (mode in-memory avec persistance)
Alternative de scaling	FAISS (IndexIVFFlat ou HNSW) pour corpus > 100 000 clauses
Métadonnées stockées	`DocumentNode` ID, juridiction, thème, date de promulgation
Stratégie de mise à jour	Ré-indexation incrémentale avec biais de récence
Risque identifié	Embedding drift entre versions anciennes et nouvelles des textes

TABLE 2.5 – Configuration du stockage vectoriel

excellente intégration avec LlamaIndex et LangChain, permettant des itérations rapides tout en conservant la possibilité de sauvegarder l'index.

Pour les déploiements à plus grande échelle (au-delà de 100 000 clauses), nous prévoyons le recours à **FAISS**. Contrairement à ChromaDB, FAISS est conçu pour gérer efficacement de très grands volumes de vecteurs, tant en termes de vitesse de recherche que de consommation mémoire. Comme l'ont démontré Johnson et al. [22], FAISS permet d'indexer et d'interroger des milliards de vecteurs avec des performances élevées.

Un aspect particulièrement important dans le contexte juridique réside dans l'exploitation des métadonnées. Chaque `DocumentNode` est enrichi avec des informations structurées (juridiction, thème, date de promulgation). Ces métadonnées permettent d'effectuer un filtrage préalable avant la phase de récupération vectorielle, réduisant ainsi les risques de contamination juridictionnelle ou temporelle.

Enfin, la stratégie de mise à jour de l'index intègre un biais de récence : lors de l'ingestion de nouveaux textes ou de versions mises à jour via le pipeline dynamique, les documents les plus récents sont priorisés lors de la ré-indexation. Cette approche vise à limiter l'impact de l'*embedding drift* entre les anciennes et nouvelles versions d'un même texte législatif.

2.3.4 Modèles LLM et embeddings

Le choix des modèles de langage et d'embeddings est guidé par la nécessité de concilier faible latence, qualité multilingue et faisabilité dans un contexte low-resource. L'architecture multi-agents itérative impose en effet des appels LLM fréquents, ce qui rend la vitesse d'inférence critique.

Pour l'extraction des clauses et l'orchestration des agents, nous utilisons les modèles Llama 3.1 via l'API Groq. Ce choix permet d'obtenir une latence très faible (inférieure à 100 ms pour le modèle Instant), indispensable au bon fonctionnement

Composant	Modèle	Raison du choix
Extraction LLM	Groq Llama-3.1-8B-Instant	Latence très faible (< 100 ms)
Orchestration multi-agents	Groq LLaMA 3.1	Même backend, cohérence et stabilité
Embeddings	HuggingFace BGE-M3	Excellentes performances multilingues (français)
Reranking	ms-marco-TinyBERT-L-2-v2	Cross-encoder léger et efficace

TABLE 2.6 – Configuration des modèles LLM et des embeddings

du workflow itératif à plusieurs agents. L'utilisation du même fournisseur pour les deux tâches garantit également une meilleure stabilité et cohérence des générations.

Concernant les embeddings, le modèle BGE-M3 a été sélectionné pour ses fortes performances en contexte multilingue, notamment sur le français juridique. Il combine des représentations denses et sparses dans un même modèle, ce qui s'avère particulièrement utile pour capturer à la fois la sémantique et les termes techniques précis (numéros d'articles, termes latins, etc.).

Le reranking est assuré par le modèle léger ms-marco-TinyBERT-L-2-v2. Ce cross-encoder permet d'affiner efficacement le classement des candidats issus de la fusion hybride (BM25 + dense + signaux de graphe) sans alourdir excessivement le temps de réponse. Son caractère compact le rend adapté aux environnements à ressources limitées tout en maintenant une bonne précision de reranking.

Ces choix permettent de maintenir un bon équilibre entre qualité des résultats, vitesse d'exécution et accessibilité, tout en restant cohérents avec les exigences de l'architecture multi-agents et de la récupération hybride.

2.3.5 BM25 et graph retrieval

Composant	Outil	Fonction
BM25	scikit-learn BM25Vectorizer	Recherche lexicale par correspondance exacte
Graph traversal	NetworkX / Neo4j (extension future)	Parcours du graphe de citations
Poids de fusion	Grid search 3D optimisé	$0.35D + 0.50B + 0.15G$

TABLE 2.7 – Composants BM25 et recherche par graphe

2.3.6 Contraintes et justification des choix technologiques

Le développement d'un système RAG avancé pour le traitement de documents juridiques dans un contexte africain francophone impose de prendre en compte plusieurs contraintes fortes : ressources computationnelles limitées, nécessité de maîtriser les coûts, et exigences de souveraineté technologique et des données. Les choix technologiques présentés dans les sections précédentes ont été guidés par ces impératifs.

Contexte low-resource et automatisation du pipeline

Le système est conçu pour fonctionner dans un environnement aux ressources limitées, tout en permettant une ingestion et un traitement des nouveaux documents de manière largement automatisée. Une fois les documents déposés dans le dossier surveillé, l'ensemble du pipeline (parsing, extraction, vectorisation, mise à jour des graphes et indexation) s'exécute sans intervention humaine.

Pour rendre cette automatisation viable dans un contexte low-resource, nous avons privilégié des solutions à la fois performantes et peu coûteuses. L'utilisation de l'API **Groq** pour les modèles Llama 3.1 permet un traitement rapide des documents entrants tout en maintenant des coûts très faibles, même lors d'une veille juridique continue. De même, le modèle d'embedding **BGE-M3** a été choisi car il est open-source et peut être exécuté localement. Cela évite de dépendre d'APIs payantes à chaque nouvelle ingestion de documents et garantit une meilleure maîtrise des coûts sur le long terme.

Ces choix techniques permettent donc de concilier deux objectifs : d'une part, un fonctionnement efficace malgré des ressources computationnelles limitées, et d'autre part, une automatisation réelle du pipeline après l'arrivée de nouveaux textes juridiques.

Justification du poids dominant de BM25

Dans la stratégie de fusion hybride, le signal **BM25** a été volontairement pondéré à 0,50, contre 0,35 pour les embeddings denses et 0,15 pour les signaux de graphe. Ce choix est motivé par les caractéristiques des textes juridiques, qui reposent fortement sur des identifiants alphanumériques précis (numéros d'articles, références à des décrets). Comme l'ont montré plusieurs études sur les corpus juridiques, BM25 reste particulièrement compétitif pour la récupération de termes exacts et de structures normatives, là où les embeddings seuls peuvent manquer de précision. Cette pondération a été validée par une recherche sur grille et s'avère robuste même avec de légères variations de poids.

Souveraineté technologique et contrôle du traitement

Bien que les documents juridiques traités soient majoritairement des textes officiels déjà publiés et accessibles sur internet, la question de la souveraineté reste pertinente dans le domaine juridique. Il s'agit moins de protéger la confidentialité des textes que de conserver le contrôle sur leur traitement et de limiter les dépendances technologiques vis-à-vis de services externes.

Nous avons donc privilégié des outils permettant un maximum de maîtrise :

— **ChromaDB** et **FAISS** peuvent être déployés localement ou sur une infrastructure maîtrisée, évitant le recours à des solutions SaaS qui imposent le transfert des données vers des serveurs tiers ;

— **LangGraph** offre une orchestration entièrement déterministe et auditable, sans dépendance à des services externes de raisonnement ou d'orchestration ;

— Le pipeline d'ingestion dynamique a été conçu pour fonctionner de manière autonome : une fois les documents déposés, le traitement (parsing, vectorisation, mise à jour des graphes) s'effectue sans transmission systématique des contenus vers des plateformes cloud.

Ces choix visent à renforcer l'indépendance technologique du système et à limiter les risques liés à la dépendance vis-à-vis d'acteurs étrangers, tout en restant compatibles avec les contraintes de coût et de performance d'un contexte low-resource.

Synthèse des arbitrages

Le tableau suivant résume les principaux arbitrages effectués entre les différentes contraintes :

Contrainte	**Choix privilégié**	**Justification**
Latence et coût	Groq + Llama 3.1	API gratuite et très rapide
Qualité multilingue	BGE-M3	Bonnes performances sur le français sans fine-tuning
Précision juridique	BM25 dominant (0,50)	Meilleure récupération des références exactes
Souveraineté	Outils open-source / locaux	Contrôle des données et auditabilité
Scalabilité future	FAISS	Capacité prouvée sur de très grands corpus

TABLE 2.8 – Arbitrages technologiques selon les contraintes du projet

Ces choix permettent de construire un système à la fois performant, économique et relativement souverain, tout en restant réaliste dans un contexte de recherche aux

ressources limitées. Ils constituent le socle technique des deux axes développés dans le chapitre suivant : le pipeline d'ingestion autonome et l'architecture multi-graphe multi-agents.

2.3.7 Architecture globale : articulation des deux axes

L'architecture du système s'articule autour de deux axes complémentaires : un pipeline d'ingestion autonome (Axe 1) et une architecture multi-graphe et multi-agents (Axe 2). Ces deux composantes sont présentées dans la figure 2.1.

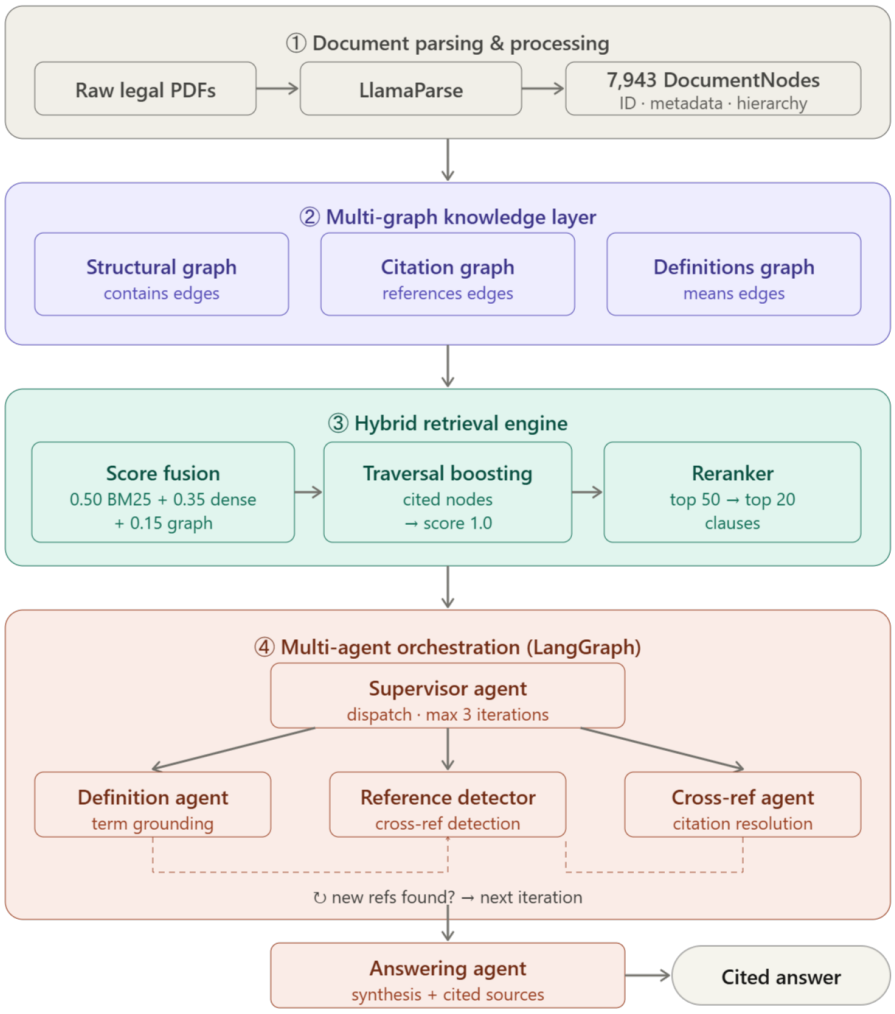


FIGURE 2.1 – Architecture globale du système RAG avancé : quatre couches intégrées couvrant le parsing des documents, la couche de graphes de connaissances, le moteur de recherche hybride et l'orchestration multi-agents via LangGraph.

L'Axe 1 vise à améliorer la précision et la traçabilité des réponses en s'appuyant sur trois graphes de connaissances (structurel, citations et définitions) et sur un moteur de recherche hybride. L'orchestration est assurée par un système multi-agents implémenté avec LangGraph, composé de cinq agents spécialisés et limité à trois itérations maximum.

L'Axe 2 répond au caractère statique des corpus RAG classiques en proposant un pipeline capable de détecter et d'intégrer automatiquement de nouveaux documents. Il repose sur un module de surveillance, un pipeline de traitement résilient, un registre de déduplication temporelle, un module de détection des conflits législatifs et un système de veille RSS autonome.

Les deux axes sont étroitement intégrés. Le pipeline d'ingestion (Axe 2) alimente en continu la base de connaissances et les graphes utilisés par le système multi-agents (Axe 1). Inversement, lors d'une requête, le système multi-agents exploite le registre de déduplication et les métadonnées produites par l'Axe 2. Cette articulation permet d'assurer à la fois la mise à jour continue des connaissances et une exploitation structurée et traçable lors du raisonnement.

Chapitre 3

Conception et Implémentation

3.1 Introduction

Ce chapitre présente la conception et l'implémentation d'un système RAG avancé adapté aux spécificités du domaine juridique, en particulier dans des contextes à ressources limitées tels que les juridictions francophones africaines et les corpus juridiques arabes. L'approche proposée repose sur deux axes technologiques complémentaires et interconnectés :

1. **Axe 1 : Architecture RAG Multi-Graphes et Multi-Agents**
   Un système de récupération et de génération augmentée par des graphes de connaissances structurés (lexical, citations et définitions), orchestré par une couche multi-agents. Cette architecture vise à garantir une traçabilité complète des sources et une précision juridique élevée, tout en permettant un raisonnement itératif sur des relations complexes entre normes.
2. **Axe 2 : Pipeline d'Ingestion Dynamique Autonome**
   Un pipeline de production capable d'ingérer, dédupliquer, analyser et mettre à jour continuellement les documents juridiques, avec détection automatique des conflits normatifs et des incohérences.

Ces deux axes s'intègrent au sein d'une architecture globale cohérente qui adresse les principales limitations des systèmes RAG classiques dans le domaine juridique : perte de la structure relationnelle des textes (citations, définitions, hiérarchie normative), difficulté à maintenir une base de connaissances à jour, et manque de traçabilité et d'explicabilité des réponses générées. La suite de ce chapitre détaille successivement la conception et l'implémentation de l'Axe 1 (sections 3.1 à 3.5), puis celle de l'Axe 2, avant de présenter leur intégration au sein du système global.

3.2 Axe 1 : Architecture RAG Multi-Graphes et Multi-Agents

3.2.1 Architecture globale

L'architecture proposée repose sur une conception intégrée en quatre couches complémentaires, conçues conjointement pour surmonter les limitations fondamentales des systèmes RAG classiques dans le domaine juridique. Ces limitations incluent notamment l'absence de traçabilité des citations, le problème de *Document Mismatch Retrieval* (DMR) et le manque de mécanismes de contrôle qualité pour garantir que chaque affirmation soit ancrée dans des sources légales précises et vérifiables.

Les quatre couches opèrent de manière séquentielle tout en intégrant une boucle récursive gérée par l'orchestration multi-agents :

La première couche assure le parsing robuste des documents juridiques bruts (PDF multi-colonnes) et leur transformation en une représentation structurée uniforme (`DocumentNode`). La deuxième couche construit trois graphes de connaissances interconnectés (lexical structurel, citations et définitions) qui capturent la hiérarchie, les dépendances légales et le vocabulaire technique. La troisième couche implémente un moteur de recherche hybride combinant signaux BM25, vectoriels denses et graphiques, suivi d'un boosting par traversal et d'un reranking de précision. Enfin, la quatrième couche orchestre l'ensemble via un système multi-agents stateful basé sur `LangGraph`, permettant une construction itérative et supervisée d'un dossier juridique complet avant toute génération de réponse.

Cette conception « citation-first » garantit que la génération de la réponse n'intervient qu'une fois le dossier certifié complet par le `Supervisor_agent`, avec traçabilité explicite de chaque source mobilisée.

3.3 Parsing et préparation des documents juridiques

Les documents juridiques africains francophones se présentent sous forme de PDF aux mises en page complexes (multi-colonnes, en-têtes, pieds de page, notes de bas de page, tableaux). Une extraction fidèle de la hiérarchie des articles, sections et chapitres est indispensable pour alimenter correctement les graphes de connaissances et permettre une granularité de récupération au niveau de la clause.

Après évaluation comparative de plusieurs parseurs sur un échantillon représentatif, `LlamaParse` a été retenu comme seule solution capable de préserver de manière fiable la hiérarchie des articles et l'intégrité du schéma. Ce choix est

corroboré par les résultats de *ParseBench* (Zhang et al., 2026), où LlamaParse Agentic obtient le score global le plus élevé (84,9 %) sur des critères critiques pour les agents : structure de tableaux, fidélité du contenu et ancrage visuel.

3.3.1 Parsing et Préparation des Documents Juridiques

Évaluation Comparative des Parseurs

Quatre parseurs ont été évalués sur un sous-ensemble représentatif de documents avec mise en page complexe (colonnes multiples, scripts mixtes, tableaux imbriqués) :

Parseur	**Hiérarchie**	**Tableaux**	**Multi-col.**	**Schéma JSON**
Docling	Partiel	Bon	Faible	Moyen
Grok	Moyen	Moyen	Moyen	Faible
Reducto	Bon	Bon	Bon	Moyen
LlamaParse	**Excellent**	**Excellent**	**Excellent**	**Excellent**

TABLE 3.1 – Comparaison des parseurs PDF. LlamaParse Agentic atteint 84,9% sur ParseBench (2000 pages vérifiées).

Justification : LlamaParse est le seul parseur à préserver systématiquement la hiérarchie des articles et l'intégrité du schéma JSON — propriété critique pour la détection de conflits.

La Figure 3.1 illustre le flux de données complet depuis le PDF brut jusqu'à la structure `DocumentNode` finale.

Schéma JSON de Contrainte et Normalisation

Chaque document parsé est converti en un objet `Document` structuré selon le schéma hiérarchique suivant :

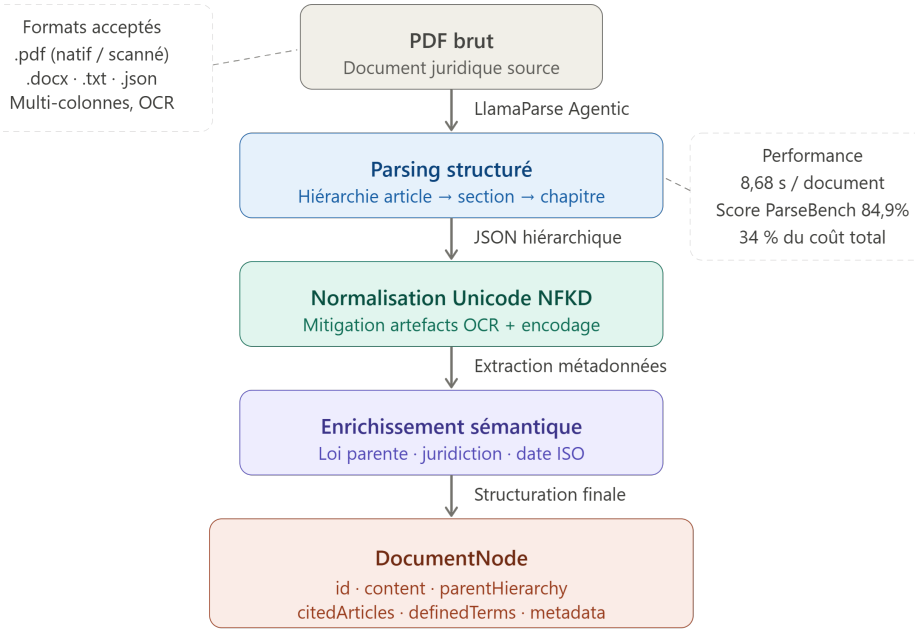


FIGURE 3.1 – Flux de données du PDF brut vers la structure `DocumentNode`. LlamaParse assure un parsing structuré de haute qualité (84,9% sur ParseBench). La normalisation Unicode NFKD réduit les artefacts OCR, tandis que l'enrichissement sémantique ajoute automatiquement la juridiction et la loi parente. Le résultat final est un `DocumentNode` normalisé et enrichi, prêt pour l'indexation et la construction des graphes.

Ce schéma est ensuite sérialisé en JSON. Voici un exemple concret de la structure produite :

```
{
  "document_id": "{Country}_{Year}_{LawName}_{ClauseID}",
  "article_number": "Article 42",
  "content": "Texte du contenu ...",
  "parent_section": "Chapter III – Protection des especes",
  "metadata": {
    "jurisdiction": "Benin",
    "theme": "Whale_Hunting",
    "promulgation_date": "2014-06-15",
    "official_title": "Loi No. 2014-19..."
  },
  "cross_references": ["Article 5", "Article 47"]
}
```

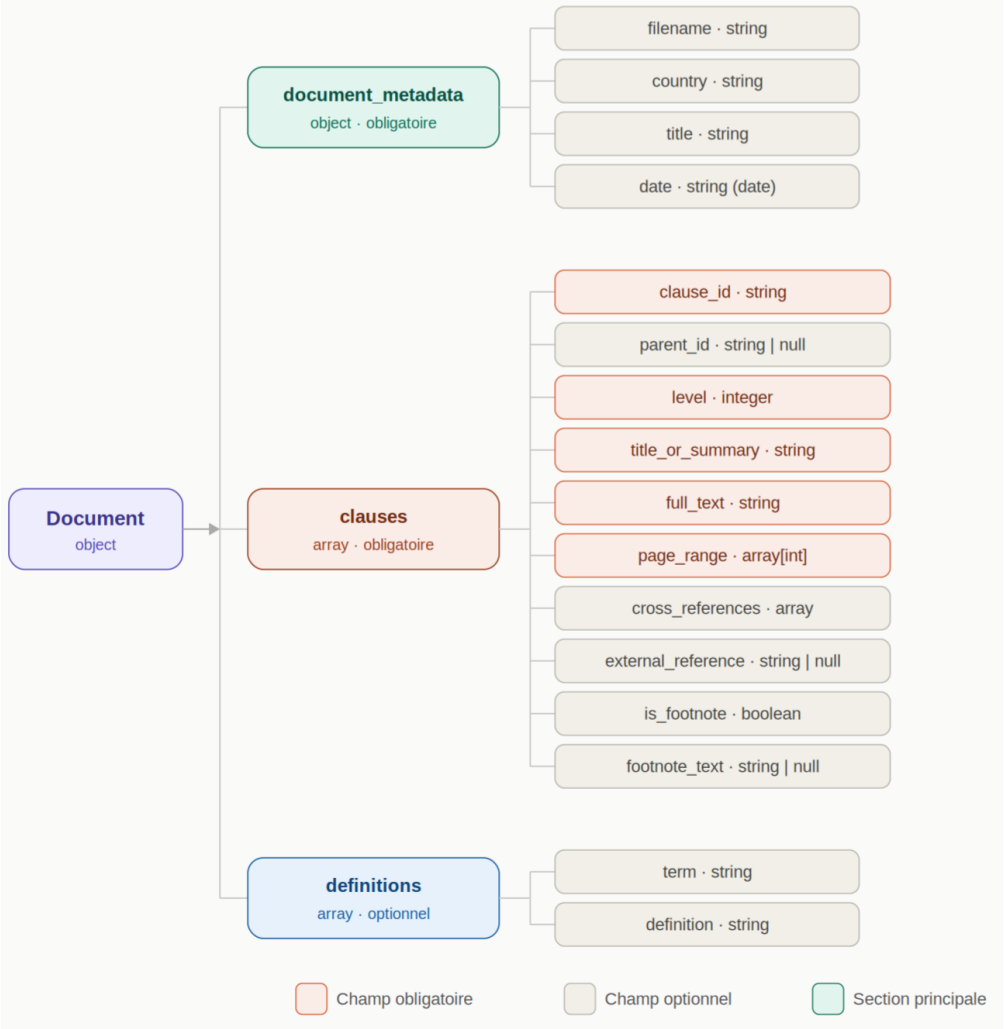


FIGURE 3.2 – Schéma hiérarchique du modèle de données `Document`.

La structure hiérarchique (via ‘parent_id‘, ‘level‘ et ‘parent_section‘) permet aux graphes de connaissances d’effectuer des traversées ascendantes et de résoudre facilement les dépendances entre articles.

Enrichissement Sémantique et Prompts d’Instruction

LlamaParse est utilisé avec un prompt d’instruction personnalisé conçu pour forcer l’extraction structurée des documents juridiques. Ce prompt demande au modèle d’extraire uniquement un objet JSON valide respectant un schéma précis, avec :

— les métadonnées du document (pays, titre, date) ;

— la hiérarchie complète des clauses (Title, Chapter, Section, Article, etc.) ;

— pour chaque clause : `clause_id`, `parent_id`, `level`, `full_text`, `cross_references`, etc. ;

— les définitions légales lorsqu'elles sont présentes.

Ce prompt joue un rôle clé dans la qualité du parsing : il garantit le respect strict de la hiérarchie et empêche la fragmentation ou la fusion incorrecte des articles. Il constitue donc la base de l'enrichissement sémantique en amont de la construction des graphes de connaissances.

Le prompt complet est disponible en annexe (Annexe A).

3.3.2 Couche de Graphes de Connaissances

Pour permettre un raisonnement récursif et une résolution déterministe des dépendances juridiques, le corpus est modélisé sous la forme de trois graphes de connaissances complémentaires. Ces graphes opèrent sur une représentation unifiée des documents .

Graphe Lexical Structurel

Ce graphe capture la hiérarchie logique interne de chaque texte normatif. Il utilise des arêtes dirigées de type `contains` pour relier chaque article à ses parents hiérarchiques (sections, chapitres et titres).

— **Nœuds** : Articles, sections, chapitres et titres.

— **Arêtes** : `contains(parent, child)`.

— **Usage** : Permet d'enrichir le contexte d'un article en remontant automatiquement vers ses parents hiérarchiques lors des requêtes.

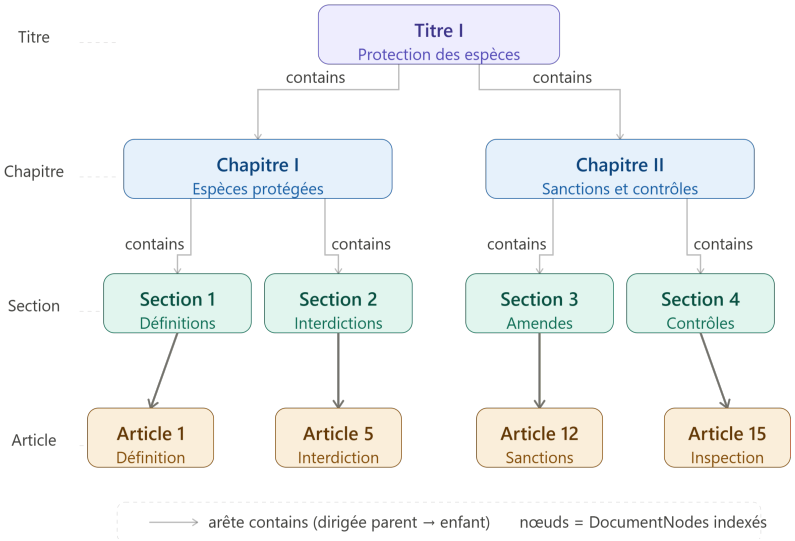


FIGURE 3.3 – Graphe structurel hiérarchique (arête `contains`). Ce graphe modélise la structure interne des documents juridiques. Chaque nœud représente un `DocumentNode` (article, section ou chapitre), et les arêtes dirigées indiquent la relation de containment.

Graphe des Citations

Ce graphe permet une résolution déterministe des dépendances légales entre les différentes dispositions normatives. Les renvois explicites présents dans les textes (tels que « conformément à l'article X » ou « visé par le Décret YY-ZZ ») sont identifiés automatiquement à l'aide d'expressions régulières. Ces renvois donnent ensuite lieu à la création d'arêtes dirigées de type `references` entre les articles concernés.

Le graphe offre ainsi trois fonctionnalités principales :

— **Extraction** : Identification automatique des renvois croisés via des expressions régulières.

— **Arêtes** : Création de liens dirigés `references(article_A, article_B)` reliant les clauses juridiquement dépendantes.

— **Traversal** : Parcours récursif du graphe permettant de récupérer l'ensemble des articles cités, auxquels est attribué un score de pertinence maximal (1,0).

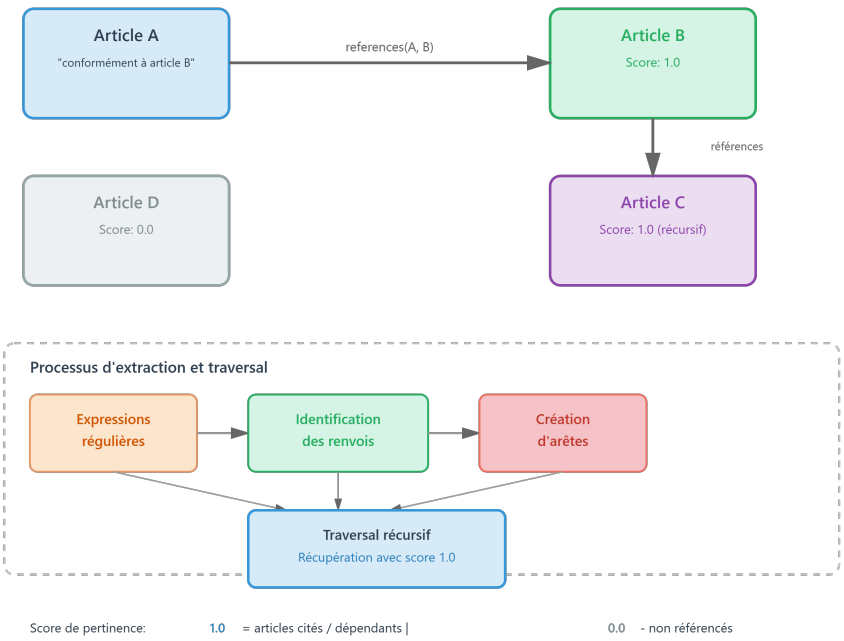


FIGURE 3.4 – Graphe des citations (`references`). Les nœuds représentent les articles juridiques et les arêtes les références explicites entre eux. Le parcours récursif permet d'inclure systématiquement les clauses dépendantes dans le contexte de récupération avec un score de pertinence maximal.

Graphe des Définitions

Ce graphe a pour objectif d'ancrer la terminologie juridique formelle. Il relie les termes techniques à leurs définitions officielles, de manière à garantir une interprétation précise et non ambiguë des textes normatifs.

Le graphe offre ainsi trois fonctionnalités principales :

— **Extraction** : Identification automatique des définitions légales présentes dans les documents.

— **Arêtes** : Création de liens de type `means(term, definition)` reliant chaque terme technique à sa définition officielle.

— **Injection** : Injection des définitions pertinentes dans le contexte avant la génération de la réponse, afin d'éviter toute ambiguïté terminologique.

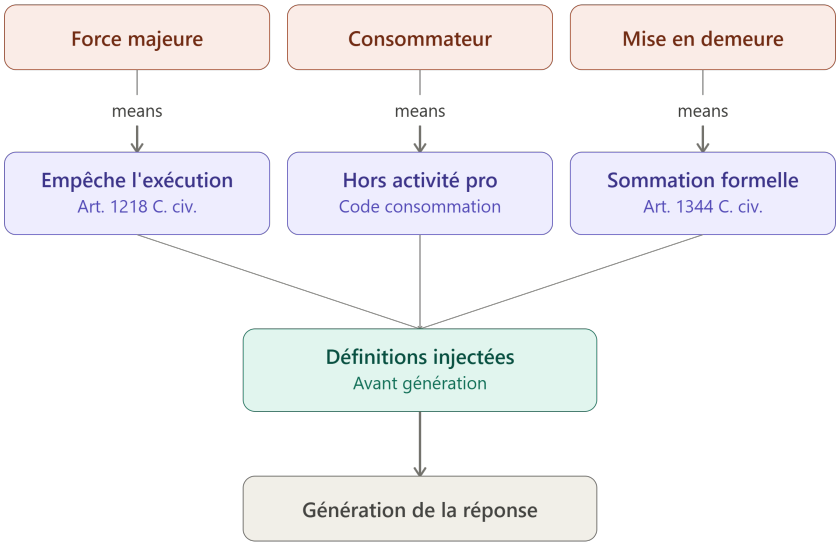


FIGURE 3.5 – Exemple de graphe des définitions (arête `means`). Chaque terme technique est relié à sa définition officielle et à l'article de référence correspondant. Ce mécanisme permet d'injecter automatiquement les définitions dans le contexte de récupération.

3.3.3 Moteur de Recherche Hybride

Le moteur de recherche hybride combine trois signaux de pertinence afin d'assurer à la fois une précision terminologique élevée et une bonne couverture sémantique.

Fusion des Scores (Dense + BM25 + Graph)

Trois récupérateurs sont exécutés en parallèle et leurs scores sont fusionnés selon l'équation suivante :

$$s_{\text{fused}} = 0.35 \cdot s_{\text{Dense (bge-m3)}} + 0.50 \cdot s_{\text{BM25}} + 0.15 \cdot s_{\text{Graph}} \tag{3.3.1}$$

— **BM25 (0,50)** : Privilégié pour l'appariement exact sur les numéros d'articles et les termes de domaine spécifiques.

— **Dense (0,35)** : Utilise l'embedding `bge-m3` pour capturer la similarité sémantique et les synonymes.

— **Graph (0,15)** : Apporte des signaux structurels issus du graphe lexical.

Les poids ont été déterminés par une recherche en grille en deux phases (recherche 3D puis 2D), permettant d'atteindre un $F_1 = 0{,}870$ sur l'échantillon de validation.

Traversal Boosting via Graphe de Citations

Après la phase de récupération initiale, un parcours récursif est effectué sur le graphe des citations. Tout `DocumentNode` explicitement référencé par un nœud du top-k voit son score de pertinence automatiquement porté à 1,0. Cette mécanisme garantit que les clauses juridiquement dépendantes ne sont jamais exclues du contexte final.

Reranking Cross-Encoder

Un funnel de récupération en deux étapes est appliqué :

— **Récupération large** : Constitution d'un pool de 50 candidats via la fusion hybride et le traversal boosting.

— **Reranking** : Les candidats sont réordonnés par un Cross-Encoder (ms-marco-TinyBERT-L-2-v2). Seuls les 20 meilleurs sont transmis à l'agent de réponse.

Robustesse des Poids (Grid Search)

Configuration	BERTScore F1	IC 95%
(0.35, 0.50, 0.15) (sélectionné)	**0.870**	[0.855, 0.885]
$(0.40, 0.45, 0.15)$ (meilleure alternative)	0.868	[0.851, 0.883]
$(0.55, 0.45, 0.00)$ (baseline)	0.843	[0.825, 0.861]

TABLE 3.2 – Robustesse des poids de fusion des scores. Les intervalles de confiance se chevauchent, confirmant la stabilité du signal BM25-dominant.

3.3.4 Orchestration Multi-Agents avec LangGraph

Le système s'appuie sur un `StateGraph` LangGraph pour orchestrer un workflow cyclique et stateful impliquant cinq agents spécialisés. L'orchestration est assurée par un `Supervisor_agent` central, qui évalue l'état du dossier après chaque étape et décide du prochain agent à activer, avec une limite stricte de 3 itérations afin d'éviter les boucles infinies.

Agents Spécialisés

Les cinq agents spécialisés sont les suivants :

1. **Supervisor_agent** : Agent central qui évalue l'état du dossier après chaque étape, dépêche l'agent suivant et impose un plafond strict de 3 itérations pour prévenir les boucles infinies.
2. **Definition_agent** : Interroge le graphe des définitions afin d'injecter les définitions officielles des termes juridiques non résolus (ex. : *hydrocarbure*, *cétacé*).
3. **Reference_detector_agent** : Analyse les clauses récupérées afin d'identifier les renvois croisés explicites et enregistre les cibles de citation non résolues pour traitement ultérieur.
4. **Cross_reference_resolution_agent** : Parcourt le graphe des citations pour résoudre les cibles en attente. Il promeut les `DocumentNode` référencés à un score de pertinence de 1,0. Le `Supervisor_agent` réévalue ensuite l'état du dossier : si de nouveaux renvois ou définitions apparaissent, un nouveau cycle est déclenché ; sinon, le contrôle est transmis à l'agent de réponse.
5. **Answering_agent** : Synthétise la réponse finale une fois que le `Supervisor_agent` a certifié la complétude du dossier. La réponse inclut obligatoirement une section « Sources consultées » listant les articles et décrets mobilisés.

Flux d'Exécution Multi-Agent

Le processus d'exécution suit le cycle itératif suivant :

1. Le `Supervisor_agent` évalue l'état initial du dossier.
2. Il active le `Definition_agent` pour injecter les définitions manquantes.
3. Il active le `Reference_detector_agent` pour détecter les renvois croisés explicites.
4. Il active le `Cross_reference_resolution_agent` pour résoudre les cibles via le graphe des citations et promouvoir les nœuds dépendants à un score de 1,0.
5. Le `Supervisor_agent` réévalue la complétude du dossier :
   - Si le dossier est complet → activation de l'`Answering_agent`.
   - Sinon → déclenchement d'une nouvelle itération (dans la limite de 3 cycles maximum).
6. L'`Answering_agent` génère la réponse finale avec une traçabilité complète des sources.

Propriété clé : Le plafond strict de 3 itérations empêche les boucles infinies tout en permettant un raffinement itératif et supervisé du dossier juridique.

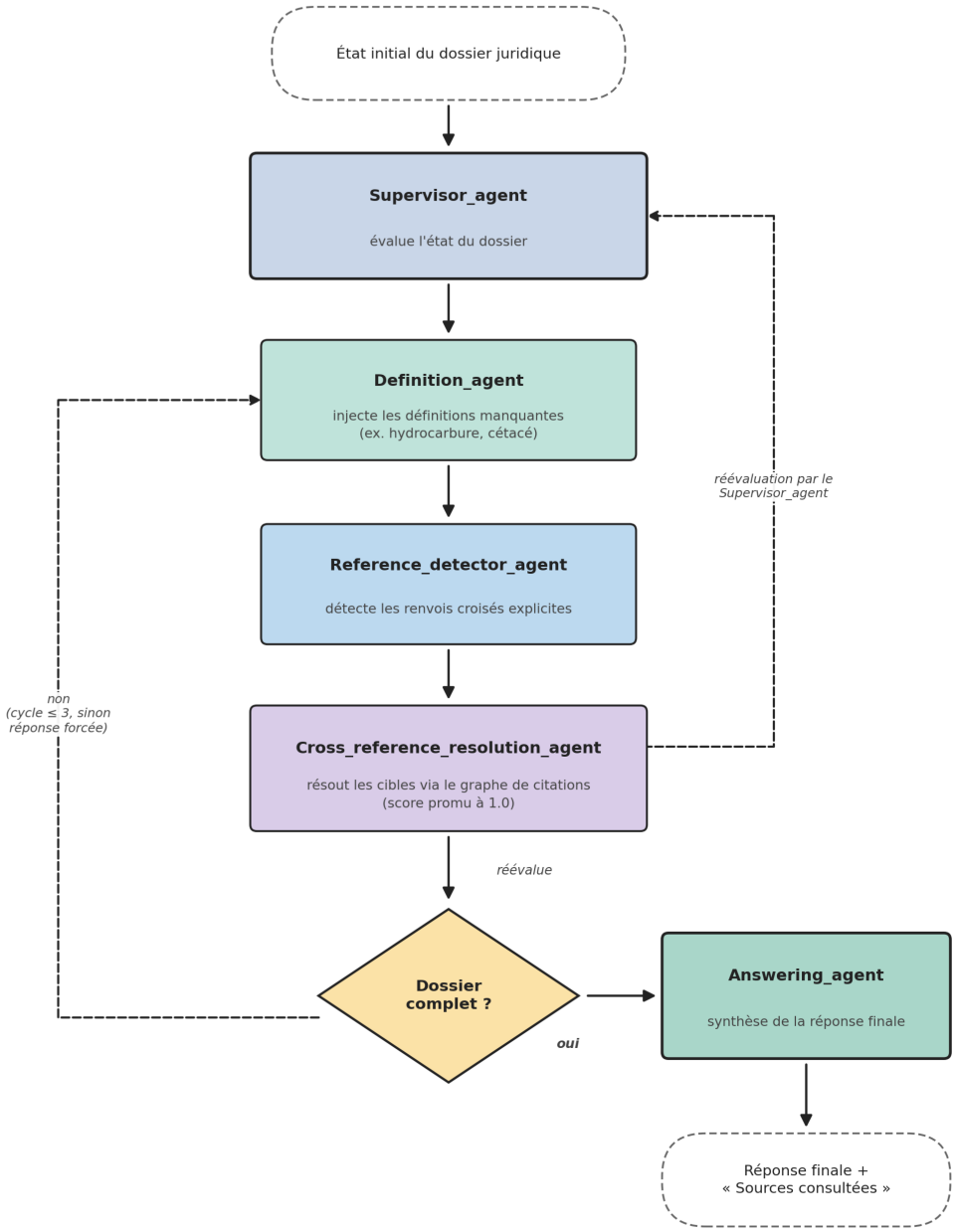


FIGURE 3.6 – Orchestration multi-agents avec LangGraph. Le `Supervisor_agent` coordonne le processus itératif en activant successivement les agents spécialisés. Après chaque cycle, il évalue la complétude du dossier juridique. L'`Answering_agent` n'est activé qu'après certification de la complétude par le superviseur. Un plafond de 3 itérations empêche les boucles infinies.

3.3.5 Diagrammes UML du système multi-agents

Pour une meilleure compréhension du fonctionnement de l'orchestration, nous présentons ci-dessous le diagramme de séquence et le diagramme d'activité du système multi-agents.

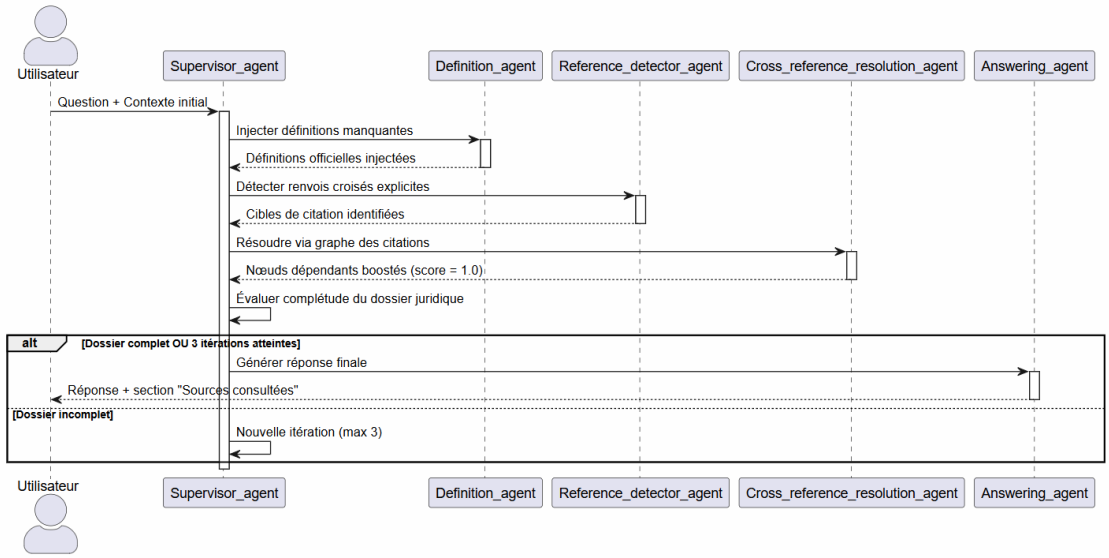


FIGURE 3.7 – Diagramme de séquence UML montrant les interactions entre les cinq agents spécialisés

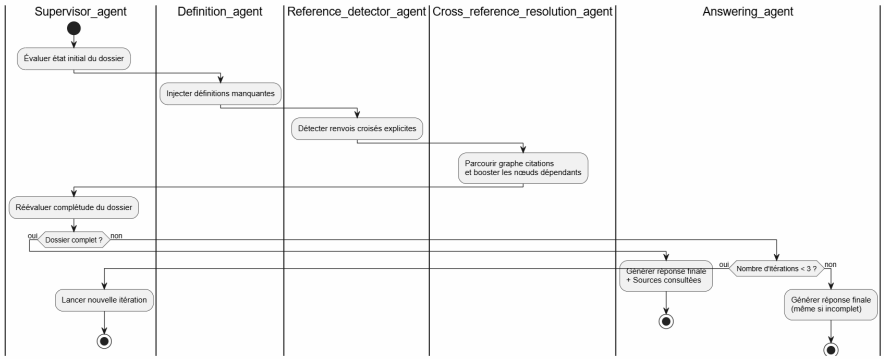


FIGURE 3.8 – Diagramme d'activité UML du processus itératif d'orchestration multi-agents avec limite de 3 itérations

3.3.6 Standard RAG (Baseline)

Afin d'évaluer les contributions de l'architecture proposée, nous définissons une baseline correspondant à un système RAG classique, que nous appelons Standard RAG.

Cette baseline partage les mêmes composants fondamentaux que notre système : les embeddings `bge-m3`, le même Cross-Encoder reranker et le même LLM (Groq LLaMA 3.1). Cependant, son pipeline est volontairement simplifié :

— Fusion des scores limitée à : $s_{\text{fused}} = 0.55 \cdot s_{\text{Dense}} + 0.45 \cdot s_{\text{BM25}}$

— Aucun graphe de connaissances (structurel, citationnel ou définitions).

— Aucun parcours récursif des renvois.

— Aucune orchestration multi-agents.

— Aucune contrainte explicite de citation imposée lors de la génération.

Ce design permet d'isoler clairement l'impact des trois graphes de connaissances et de l'orchestration multi-agents sur les performances finales.

3.4 Axe 2 : Pipeline d'Ingestion Dynamique Autonome

3.4.1 Architecture du pipeline d'ingestion autonome

Pour répondre à la problématique de l'obsolescence rapide des bases de connaissances juridiques, nous intégrons dans notre système une couche d'ingestion dynamique et autonome appelée **LegalStreamRAG**. Cette couche vient compléter l'architecture de récupération et de raisonnement développée dans l'Axe 1.

L'Axe 2 poursuit deux objectifs principaux :

— Assurer l'intégration continue et automatisée des nouveaux documents juridiques sans reconstruction complète de la base de connaissances ;

— Garantir la cohérence temporelle du corpus en détectant et en gérant les évolutions législatives (modifications, abrogations implicites, conflits de normes).

Le pipeline est organisé en deux phases complémentaires :

— **PHASE A** (hors-ligne) : ingestion, structuration et détection de conflits ;

— **PHASE B** (en-ligne) : exploitation des données mises à jour par les requêtes utilisateurs via le moteur hybride et l'orchestration multi-agents.

3.4.2 PHASE A : Ingestion et Structuration (hors-ligne)

Cette phase est déclenchée automatiquement dès l'arrivée d'un nouveau document dans le dossier `data/input/` (via le composant `watcher.py`).

Détection et déclenchement

Le module `NewFileHandler` (dans `watcher.py`) surveille le dossier d'entrée. Dès la création d'un fichier, il patiente 2 secondes pour s'assurer que le transfert est terminé, puis lance le pipeline complet via `LegalDocumentParser.parse_file`.

Parsing, structuration et enrichissement

Le document est traité selon les étapes suivantes :

1. **Parsing structuré** avec LlamaParse pour obtenir une représentation hiérarchique (titres, chapitres, sections, articles) au format Markdown ;
2. **Extraction IA** via Groq (`llama-3.1-8b-instant`) selon le schéma JSON défini dans l'Axe 1 ;
3. **Nettoyage et normalisation** (recollage des mots coupés, fusion des fragments courts) ;
4. **Création des DocumentNode** enrichis et prêts pour l'indexation.

Mise à jour incrémentale des graphes de connaissances

Les relations extraites du nouveau document sont fusionnées de manière incrémentale dans les fichiers globaux `lexical_graph.json` et `definitions_graph.json`. Le document original est ensuite archivé dans `data/archive/` pour garantir la traçabilité.

3.4.3 Gestion de l'évolution et de la cohérence temporelle

L'un des apports majeurs de l'Axe 2 est la détection automatique des conflits et des évolutions législatives lors de l'ingestion d'un nouveau document.

Processus de détection de conflits documentaires

Deux processus complémentaires ont été conçus pour identifier et traiter les contradictions entre le nouveau document et la base de connaissances existante.

Le processus global se décompose en cinq étapes principales :

1. **Recherche dans la base de données** : identification des documents déjà traités du même pays dans `data_processed/` ;

2. **Sélection des candidats** : comparaison des articles du nouveau document avec ceux des documents existants ;
3. **Filtrage par mots-clés** : sélection des paires d'articles partageant plus de 10 mots significatifs ;
4. **Résolution par LLM** : soumission des paires candidates à Groq pour détecter une contradiction, une incompatibilité ou une abrogation implicite ;
5. **Génération du rapport** : enregistrement des conflits dans un fichier `*_conflicts.json` et notification automatique (Markdown + HTML) via `src/notifier.py`.

Typologie des conflits détectés

Trois grandes catégories de contradictions peuvent être identifiées lors de l'ingestion :

— **Conflit temporel (abrogation tacite)** : une nouvelle disposition modifie ou remplace implicitement une règle antérieure (ex. : modification des amendes ou des durées) ;

— **Conflit de spécialité** : une norme sectorielle contredit une disposition plus générale du corpus (*lex specialis derogat legi generali*) ;

— **Conflit de hiérarchie des normes** : un texte national entre en contradiction avec une convention internationale déjà intégrée (ex. : MARPOL, CITES).

Limites actuelles et résilience du système

L'implémentation actuelle de la fonction `detect_legal_conflicts` présente certaines limitations techniques (limitation aux 3 premiers articles longs, filtrage lexical simpliste, filtre strict par pays). Ces limites sont connues et le système privilégie actuellement la détection des conflits les plus évidents, tout en signalant systématiquement les cas détectés à un juriste via le module de notification pour validation humaine (mise à jour du champ `valid_to` ou versioning).

3.4.4 Veille juridique automatique et mise à jour continue

Le module de veille, orchestré par `src/scheduler.py` et la classe `LegalMonitor`, interroge de manière proactive des sources officielles (flux RSS, Google News, sites gouvernementaux). Lorsqu'une mise à jour critique est détectée, elle est convertie en document structuré et déposée automatiquement dans le dossier d'entrée pour être traitée par la PHASE A. Des rapports de veille au format Markdown et HTML sont générés et visibles sur le tableau de bord.

3.4.5 PHASE B : Recherche Hybride et Orchestration Multi-Agent (en-ligne)

Cette phase s'active lors d'une requête utilisateur. Elle exploite les graphes mis à jour par la PHASE A et les documents intégrés via la veille automatique. Le processus (recherche hybride, boosting topologique, boucle multi-agents et génération citée) est détaillé dans l'Axe 1.

3.4.6 Synthèse et intégration des deux axes

L'intégration des Axes 1 et 2 forme un système complet et cohérent :

— L'**Axe 1** garantit la qualité, la traçabilité et la complétude contextuelle des réponses grâce aux graphes de connaissances et à l'orchestration multi-agents ;

— L'**Axe 2** assure la fraîcheur et la cohérence temporelle du corpus via l'ingestion autonome, la détection de conflits et la veille juridique automatique.

L'intégration technique repose sur le registre de déduplication temporelle et sur le dépôt automatique des documents issus de la veille. Le surcoût observé reste acceptable (latence moyenne de 8,1 secondes pour les requêtes complexes).

Des scripts de test (`test_advanced_features.py`, `test_ingestion_conflict.py`, `test_conflict_notification.py`) ont été développés pour valider le bon fonctionnement de chaque composant du pipeline.

Chapitre 4

Expérimentations et Résultats

4.1 Protocole Expérimental

4.1.1 Corpus et Benchmark

Le corpus comprend des textes statutaires et réglementaires en français (*lois*, *décrets*, *ordonnances*) gouvernant la protection environnementale, particulièrement :

— Régulation de la chasse aux baleines

— Protection des oiseaux marins

— Contrôle des rejets d'hydrocarbures

Ces documents proviennent de juridictions francophones africaines et de la convention internationale MARPOL.

Composition du Corpus

La base de connaissances comprend 50 PDFs uniques couvrant 17 juridictions :

Thème	**Documents**	**Clauses**
Whale Hunting	23	2,080
Marine Birds	14	957
Hydrocarbon Discharge	21	4,906
Total	**50**	**7,943**

TABLE 4.1 – Statistiques du corpus par thème. Tous textes en français.

Juridiction	Clauses	Juridiction	Clauses
Algérie	621	Madagascar	374
Bénin	418	MARPOL	712
Cameroun	593	Mauritanie	297
Comores	284	Maroc	538
Congo	310	Sénégal	612
Côte d'Ivoire	452	Togo	287
Djibouti	195	Tunisie	397
France	489		
Gabon	601		
Guinée	263		

TABLE 4.2 – Distribution clauses par juridiction (50 documents uniques = 7,943 clauses).

Benchmark et Questions-Réponses Validées

Le benchmark d'évaluation comprend **263 paires question-réponse** validées par experts en droit statutaire francophone africain.

— **Whale Hunting** : 131 paires Q&R

— **Hydrocarbon Discharge** : 132 paires Q&R

— **Marine Birds** : Exclues de benchmark (annotation non complétée), incluses en corpus pour couverture écologique.

Pour chaque question, les experts ont fourni :

1. Réponse de référence (ground truth)
2. Articles autoritaires requis pour supporter la réponse
3. Établissement d'un standard d'or objectif pour exactitude des citations

4.1.2 Métriques d'Évaluation

Métriques RAGAS

Framework RAGAS (*Retrieval-Augmented Generation Assessment*) évalue cinq dimensions clés (scores 0–1) :

1. **Faithfulness** : Réponse générée fidèle au contexte récupéré (pas d'hallucinations).
2. **Answer Relevancy** : Réponse directement pertinente à la question posée.
3. **Context Recall** : Contexte récupéré couvre tous articles pertinents en ground truth.

4. **Answer Correctness** : Réponse match ground truth expert.
5. **Answer Similarity** : Similarité sémantique entre réponse générée et ground truth.

Note : Context Precision exclu car requiert labels ground-truth au niveau chunk, non disponibles dans protocole annotation actuel.

BERTScore

Score F1 pairwise entre baseline RAG, système multi-agent, et ground truth expert, calculé via embeddings `bge-m3`. Vérification supplémentaire avec `multilingual-e5-large` pour exclure inflation induite embeddings.

Exactitude des Citations (Métrique Binaire)

Pour chaque requête : système a-t-il cité l'article légal exact correspondant au ground truth expert ?

— Réponse = Correct / Incorrect

— Métrique critique hard pour applications légales haute-stakes

Signification Statistique

Intervalles de confiance bootstrap 95% et tests de Wilcoxon signed-rank appariés ($\alpha = 0.05$). Contamination croisée entre thèmes prévenue via partitionnage métadonnées des indices au temps requête.

4.2 Résultats de l'Architecture RAG Multi-Graphes et Multi-Agents

4.2.1 Métriques RAGAS

L'amélioration la plus significative observée porte sur le **Context Recall**, qui augmente fortement sur les deux thématiques évaluées. Ce résultat confirme que le parcours récursif des citations, rendu possible par les graphes de connaissances, permet de récupérer un contexte beaucoup plus complet que les approches classiques de récupération en une seule passe.

Système	Faith.	Ans.R	C.R	A.Cor	A.Sim
Thème 1 : Whale Hunting (n=131)					
Naive RAG	0.41	0.51	0.22	0.14	0.18
Multi-Agent	0.83	0.84	0.86	0.78	0.85
Thème 2 : Hydrocarbon Discharge (n=132)					
Naive RAG	0.39	0.48	0.19	0.12	0.16
Multi-Agent	0.88	0.87	0.89	0.90	0.80

TABLE 4.3 – Scores RAGAS par thème.

4.2.2 BERTScore et Étude d'Ablation

Le tableau ci-dessous présente le BERTScore F1 poolé sur l'ensemble du benchmark d'évaluation, c'est-à-dire sur les 263 paires question-réponse validées par des experts en droit statutaire francophone africain. Le thème « Marine Birds » a été exclu du calcul en raison de l'absence de paires Q&R annotées.

Comparaison	BERTScore F1	IC 95%
Baseline vs Référence	0.377	[0.358, 0.394]
Multi-Agent vs Référence	**0.844**	[0.825, 0.863]

TABLE 4.4 – BERTScore F1 poolé sur les deux thématiques évaluées (n=263).

L'étude d'ablation réalisée sur le thème Whale Hunting met en évidence une interaction super-additive entre la couche de graphes de connaissances et l'orchestration multi-agents.

Mode	BERTScore F1	IC 95%	Exactitude des Citations
No-Graph	0.559	[0.530, 0.588]	71.97%
No-Agents	0.579	[0.551, 0.607]	60.61%
Full System	**0.882**	**[0.861, 0.903]**	**89.3%**

TABLE 4.5 – Étude d'ablation — Thème Whale Hunting (n=131). Les intervalles de confiance non chevauchants confirment la significativité statistique.

Interprétation : Le retrait de la couche de graphes entraîne une baisse significative de -0.323 en BERTScore F1, tandis que le retrait de l'orchestration multi-agents provoque une baisse de -0.303. Seul le système complet atteint le pic de performance. Ces résultats démontrent que les graphes de connaissances et les

agents ne contribuent pas de manière additive, mais **synergique** : leur combinaison produit un effet supérieur à la somme de leurs contributions individuelles. Cela confirme l'importance de l'architecture intégrée proposée dans ce travail.

4.2.3 Exactitude des Citations

Thème	**Multi-Agent**	**Naive RAG**
Whale Hunting (n=131)	96.2%	27.5%
Hydrocarbon Discharge (n=132)	91.7%	24.2%
Poolé (n=263)	**93.9%**	**25.9%**

TABLE 4.6 – Exactitude des citations (métrique binaire).

Ces résultats ont été confirmés par une évaluation réalisée à l'aide du modèle de langage Claude sur un échantillon stratifié de 60 questions (30 par thème). L'exactitude de citation obtenue s'élève à 93.3% pour le système Multi-Agent, contre 26.7% pour la baseline Standard RAG.

4.2.4 Exemples Qualitatifs Contrastés

Afin d'illustrer concrètement la différence de qualité entre les deux systèmes, le tableau ci-dessous présente deux requêtes représentatives.

Observation : La baseline génère des réponses vagues sans référence précise. Le système multi-agents fournit des réponses précises, avec les articles exacts et les sources consultées.

Requête	Naive RAG (Baseline)	Système Multi-Agent
Sanctions pour la chasse aux cétacés au Bénin ?	La chasse au cétacé est réglementée par le droit environnemental béninois.	**Article 97 de la Loi 87-014** prévoit une amende de 500 000 FCFA et une peine d'emprisonnement de 1 à 3 ans. **Sources** : Loi 87-014, Art. 97 ; Décret 2000-278, Art. 4.
Décret régissant les rejets d'hydrocarbures au Gabon ?	Les rejets sont régis par la législation MARPOL.	Le **Décret 0163/PR/MEPN du 3 février 2010, Article 12** établit les seuils autorisés, sur la base de l'**Article 23 du Code de la Pêche** du Gabon. **Sources** : Décret 0163, Art. 12 ; Code de la Pêche Gabon, Art. 23.

TABLE 4.7 – Exemples contrastés de réponses générées.

4.3 Résultats du Pipeline d'Ingestion Dynamique

4.3.1 Performance End-to-End

Les performances du pipeline d'ingestion ont été testées sur un document juridique de 75 pages. Le tableau ci-dessous présente les temps de traitement observés pour les différentes étapes du pipeline.

Étape	Temps moyen	Temps total
PDF Parsing (LlamaParse)	8.68 s	399.5 s
LLM Extraction (Groq)	15.53 s	714.4 s
Post-processing & RAG	1.23 s	56.4 s
Total	-	**19.5 min**

TABLE 4.8 – Performance du pipeline d'ingestion sur un document de 75 pages.

Le pipeline a permis de traiter l'intégralité du document en 19 minutes et 30 secondes. Ce temps de traitement reste acceptable pour un usage en conditions réelles, notamment dans le cadre d'une ingestion automatique et planifiée.

4.3.2 Résilience et Robustesse

Testé sous forte contrainte d'API (limites de tokens/minute de Groq), le pipeline a traité un contrat de 6 pages (172 clauses) en 9 minutes malgré 10 rejets HTTP 429, gérés par exponential backoff. Le taux de réparation automatique des JSON a atteint 100%. Ces résultats confirment la robustesse du système en conditions réelles.

4.3.3 Efficacité de la Veille Juridique et de la Déduplication

Le module de veille automatique a été évalué sur plusieurs cycles de surveillance des sources juridiques. Les performances observées sont les suivantes :

— Scan complet de 5 sources RSS : **22 secondes** ;

— Scan de 8 sources avec fallback HTML (dont Adala) : **95 secondes** ;

— Précision du filtrage de pertinence : **83.3%** ;

— Réduction de la charge de traitement grâce au mécanisme de déduplication : **86.1%**.

Ces résultats montrent l'efficacité du système à la fois pour la détection des mises à jour juridiques et pour la limitation de la charge de traitement inutile.

4.4 Discussion

4.4.1 Apports Principaux

L'évaluation du système proposé met en évidence plusieurs contributions significatives :

1. **Amélioration majeure de la complétude contextuelle** : Le système multi-agents permet d'atteindre un *Context Recall* de 0.86–0.89, contre 0.19–0.22 pour la baseline Naive RAG. Cette amélioration (+291 % à +368 %) démontre l'efficacité du parcours récursif des citations rendu possible par les graphes de connaissances.
2. **Exactitude de citation professionnelle** : Le système atteint 93,9 % d'exactitude de citation (validé à 93,3 % par évaluation humaine avec $\kappa = 0,81$). Ce niveau de traçabilité répond aux exigences du domaine juridique, contrairement à la baseline (25,9 %).
3. **Synergie entre graphes et agents** : L'étude d'ablation met en évidence une interaction super-additive entre la couche de graphes de connaissances et l'orchestration multi-agents. Le retrait de l'un ou l'autre composant entraîne

une baisse significative des performances, alors que le système complet atteint un niveau nettement supérieur à la somme des contributions individuelles. Cela confirme que les deux composants fonctionnent de manière complémentaire.

4. **Infrastructure d'ingestion viable en contexte réel** : Le pipeline d'ingestion autonome démontre une bonne résilience (100 % d'auto-réparation JSON), un coût très faible (0,15 $ pour 46 documents) et une efficacité de veille juridique satisfaisante (83,3 % de précision de filtrage).

Ces résultats confirment l'hypothèse de départ selon laquelle une architecture combinant graphes de connaissances et orchestration multi-agents permet d'améliorer significativement la qualité des réponses juridiques par rapport aux approches RAG classiques.

4.4.2 Limites et Axes d'Amélioration

Malgré des résultats encourageants, plusieurs limites persistent et constituent des axes d'amélioration clairs pour les travaux futurs :

— **Erreur temporelle (ERR-2.2)** : Le système peut encore récupérer des versions obsolètes d'un texte légal lorsque plusieurs versions d'une même disposition coexistent. Cette limite, identifiée comme l'erreur la plus fréquente lors de l'analyse qualitative, constitue le principal axe d'amélioration pour les travaux futurs. Une détection explicite de la précédence temporelle entre versions serait nécessaire pour garantir la cohérence des réponses dans le temps.

— **Latence des requêtes complexes** : Le temps de réponse moyen s'élève à 8,1 secondes pour les requêtes complexes. Ce délai reste supérieur à l'objectif de temps réel (< 2 secondes) visé pour certains usages interactifs.

— **Scalabilité** : L'architecture a été validée sur un corpus de taille moyenne (7 943 clauses). Un déploiement sur des corpus significativement plus volumineux nécessiterait des optimisations supplémentaires, notamment en matière de compression de contexte et d'indexation hiérarchique des graphes.

— **Couverture de détection des conflits** : Les contraintes de conception (troncature des candidats, filtrage lexical strict) limitent actuellement le nombre de conflits détectés par rapport au potentiel théorique. Ces choix ont été faits pour maîtriser la complexité dans le cadre d'un mémoire de Master.

Chapitre 5

Conclusion Générale

Ce mémoire de fin d'études a porté sur la conception et l'évaluation d'un système RAG avancé spécialisé dans le traitement des documents juridiques francophones, avec un focus particulier sur l'expansion sémantique des articles et la gestion dynamique des évolutions législatives.

5.1 Rappel de la problématique et des objectifs

Les systèmes RAG classiques présentent des limites structurelles importantes lorsqu'ils sont appliqués au domaine juridique : perte de contexte liée au chunking séquentiel, incapacité à résoudre les références croisées, et absence de mécanisme de mise à jour temporelle des connaissances. Ces faiblesses entraînent des hallucinations, des réponses incomplètes et une faible traçabilité des sources, ce qui est particulièrement problématique dans un domaine où la précision et la fiabilité sont essentielles.

Pour répondre à cette problématique, ce travail s'est structuré autour de deux axes complémentaires :

— **Axe 1** : La conception d'une architecture RAG multi-graphes et multi-agents capable d'effectuer une expansion sémantique intelligente des articles juridiques.

— **Axe 2** : Le développement d'un pipeline d'ingestion autonome et résilient permettant la mise à jour continue du corpus tout en assurant la cohérence temporelle des connaissances.

5.2 Synthèse des apports

Sur le plan **conceptuel**, ce travail a proposé une modélisation originale des documents juridiques à travers trois graphes complémentaires (structurel, des citations et des définitions). Cette approche permet de capturer les relations complexes propres au droit et d'enrichir chaque article avec son contexte complet.

Sur le plan **architectural**, l'intégration d'une couche d'orchestration multi-agents via LangGraph, combinée à un moteur de recherche hybride (dense + sparse + graphe), a permis d'atteindre un niveau de discipline de citation et de traçabilité rarement observé dans les systèmes RAG juridiques existants.

Sur le plan **empirique**, les expérimentations menées sur un corpus de 50 documents juridiques (7 943 clauses) couvrant 17 juridictions francophones africaines ont démontré des gains substantiels par rapport à une baseline Naive RAG. Le système a notamment atteint **93,9 %** d'exactitude de citation et des scores RAGAS significativement supérieurs sur les dimensions de *Context Recall* et *Answer Correctness*. Par ailleurs, le pipeline d'ingestion autonome a montré sa viabilité opérationnelle avec un temps moyen de traitement de 25,44 secondes par document et une excellente résilience face aux limites d'API.

5.3 Réponse à la question de recherche

Les résultats obtenus confirment l'hypothèse principale de ce travail : l'intégration d'un mécanisme hybride de résolution de références croisées (règles + LLM + graphes de connaissances), combinée à une expansion sémantique des articles et à un pipeline d'ingestion dynamique conscient des évolutions législatives, permet d'obtenir une amélioration significative de la fidélité contextuelle et de la réduction des hallucinations par rapport aux approches RAG classiques.

5.4 Limites du travail

Ce travail présente néanmoins plusieurs limites. La principale concerne la gestion des versions temporelles des textes légaux, qui reste perfectible. Par ailleurs, l'architecture a été validée sur un corpus de taille moyenne ; son passage à l'échelle sur des corpus plus volumineux nécessiterait des optimisations supplémentaires. Enfin, bien que les résultats soient encourageants, une évaluation sur d'autres domaines du droit (droit des affaires, droit administratif, etc.) serait nécessaire pour généraliser les conclusions.

5.5 Perspectives de recherche

Plusieurs pistes de recherche futures se dégagent de ce travail. La première consiste à approfondir le versioning fin des dispositions légales (*clause-level versioning*) afin de mieux gérer les abrogations et modifications partielles. Une deuxième perspective porte sur l'extension du système aux corpus multilingues, notamment arabophones et lusophones, particulièrement pertinents dans le contexte africain. Enfin, l'intégration de mécanismes de raisonnement temporel et de détection automatique des conflits normatifs constitue un axe prometteur pour renforcer encore la fiabilité des systèmes RAG juridiques.

Annexe A

Prompts des agents et du pipeline d'ingestion

Cette annexe regroupe l'ensemble des prompts utilisés dans le système RAG multi-agents, notamment le prompt de parsing avec LlamaParse et les différents agents du système.

A.1 Prompt de parsing LlamaParse

Listing A.1 – Prompt complet utilisé avec LlamaParse pour l'extraction structurée des documents juridiques

```
You are an expert in legal document parsing and information
    extraction.
Your task is to extract the complete hierarchical structure of a
    legal document and output ONLY a valid JSON object following
    the schema described below.

Schema
document_metadata (required)
Extract:
- filename : string
- country : string
- title : string
- date : string in format YYYY-MM-DD

clauses (required)
Extract ALL clauses and preserve the exact hierarchy of the
    document.
Possible hierarchy levels include: Book, Part, Title, Chapter,
    Section, Subsection, Article, Paragraph, Item, Point, Annex.
```

```
For each clause extract:
- clause_id : string (Official identifier exactly as written)
- parent_id : string or null
- level : integer (1 = root, 2 = child of root, etc.)
- title_or_summary : string
- full_text : string (Preserve the original text exactly)
- page_range : array of integers
- cross_references : array of strings
- external_reference : string or null
- is_footnote : boolean
- footnote_text : string or null

definitions (optional)
If the document contains definitions, extract:
- term : string
- definition : string

Important rules:
1. Preserve the original language.
2. Preserve numbering exactly.
3. Preserve the hierarchy exactly.
4. Do NOT merge clauses.
5. Do NOT invent information.
6. Return null when information is missing.
7. Return [] for empty arrays.
8. Return ONLY JSON.
9. Do NOT output markdown.
10. Do NOT output explanations.

Output format:
{
  "document_metadata": {},
  "clauses": [],
  "definitions": []
}
```

A.2 Prompts du système multi-agents

A.2.1 Agent Routeur (router_agent)

System Prompt :

```
Tu es un agent intelligent supervisant un processus de
    r cup ration multi-agent dans un graphe de connaissances
    juridiques.
Tu dois d terminer si les documents r cup r s suffisent pour
    r pondre    la requ te , ou s'il faut r cup rer des
```

```
   informations suppl mentaires.
Analyse les documents ci-dessous et d cide :
- STOP : les documents suffisent pour r pondre    la requ te
- RECURSE : des r f rences internes doivent  tre  r cup r es.
    Liste les clause_id ou node_id    r cup rer.
- EXTERNAL : des r f rences    des lois ou codes externes
   doivent  tre  recherch es.
R ponds au format JSON :
{"decision": "STOP"|"RECURSE"|"EXTERNAL", "node_ids": [...], "
   footer_queries": {...}, "external_refs": [...]}
```

User Prompt :

```
Requ te : {query}
Documents r cup r s :
{context}
D finitions disponibles : {list(state.get('definitions', {}).keys
   ())}
Analyse les documents et prends une d cision. R ponds UNIQUEMENT
    en JSON.
```

A.2.2 Agent de Réponse (answering_agent)

System Prompt :

```
Tu es un agent de synth se juridique. Tu dois r pondre    la
   requ te de l'utilisateur en te basant EXCLUSIVEMENT sur les
   documents, d finitions et r f rences fournies.
Instructions :
1. R ponds de mani re structur e et compl te
2. Cite syst matiquement les articles/clauses sources
3. Utilise les d finitions fournies pour contextualiser la
   r ponse
4. Int gre les r f rences externes si pr sentes
5. Ne pas inventer d'informations
6. R ponds en fran ais
7. Si le contexte est insuffisant, indique-le explicitement
```

User Prompt :

```
Requ te : {query}
{full_context}
R dige une r ponse compl te et structur e avec citations des
   sources.
```

A.2.3 Super-Agents (scripts/super_agents.py)

— **Agent Définition (definitions_agent)**

```
Extrait uniquement les concepts juridiques cl s de cette
    requ te qui n cessitent une d finition stricte : '{
    state.query}'. Renvoie-les sous forme de liste JSON.
```

— **Agent Routeur (router_agent)**

```
Tu es un agent routeur. Voici le contexte r cup r  pour
    r pondre    : {state.query}
{context}
D termine si tu dois r cup rer d'autres articles de loi
    cit s dans ce contexte.
R ponds uniquement par une liste de r f rences    chercher
    , ou None si l'info est suffisante.
```

— **Agent Superviseur (supervisor_agent)**

```
Tu es superviseur. Question : {state.query}.
Voici le contexte : {context}.
Si on a assez d'infos, r ponds END. Sinon CONTINUE.
```

— **Agent de Réponse final (answering_agent)**

```
Tu es un expert juridique. R ponds EXCLUSIVEMENT    partir
    des sections suivantes.
Cite les articles de loi. Contexte :
{context}
Question : {state.query}
```

A.3 Captures d'écran des interfaces du système

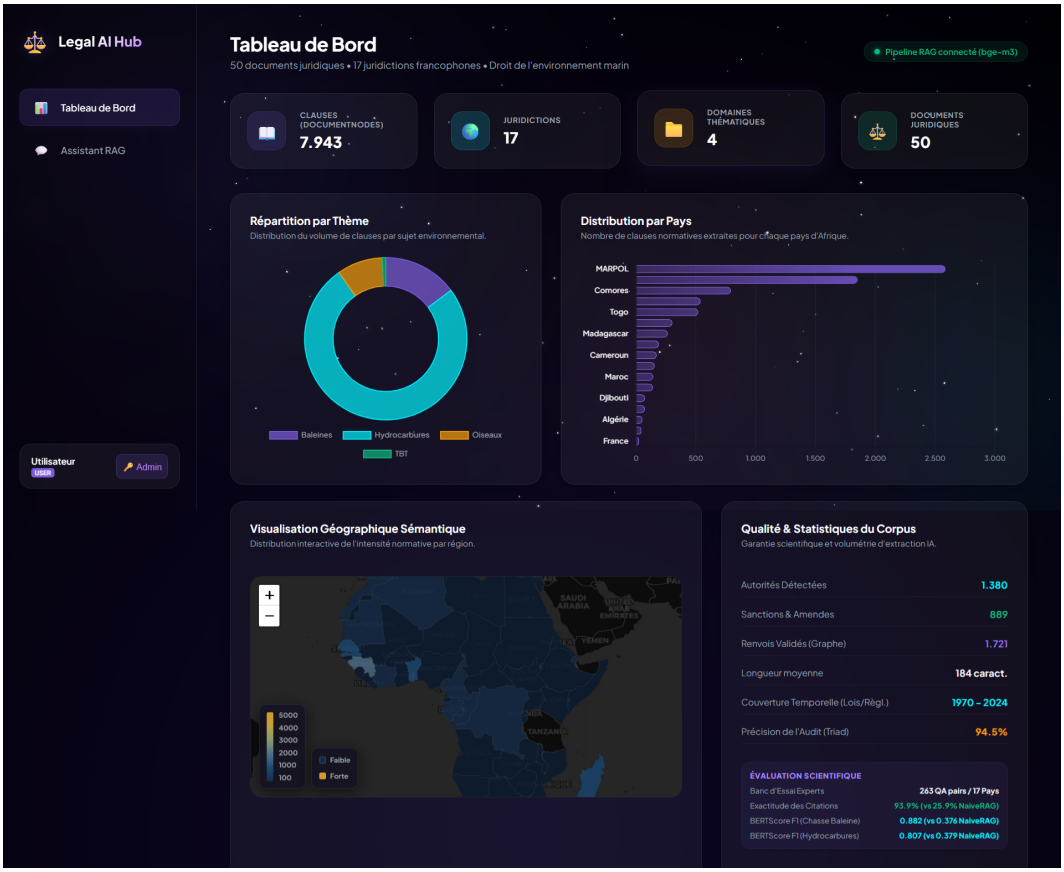


FIGURE A.1 – Interface principale du site juridique

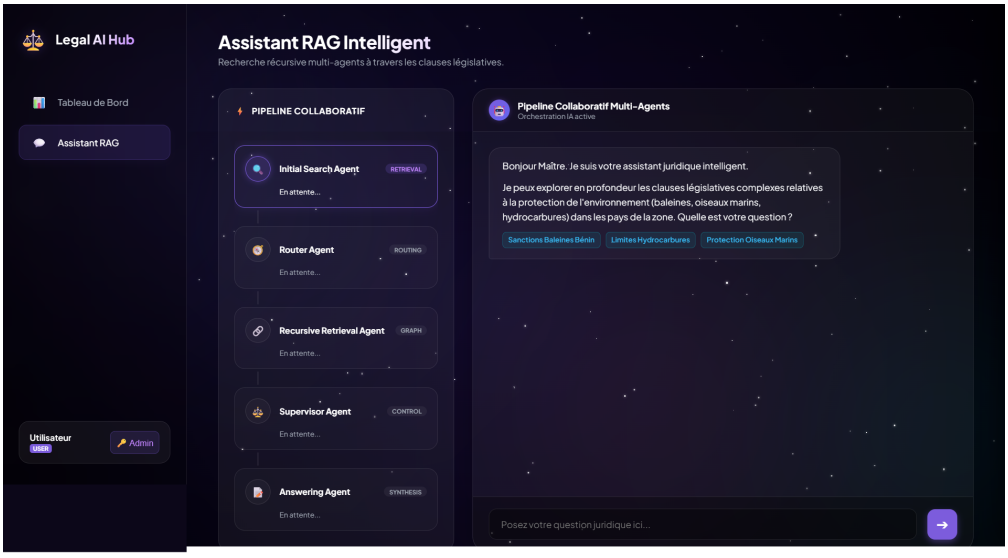


FIGURE A.2 – Assistant RAG Intelligent – Interface de question-réponse

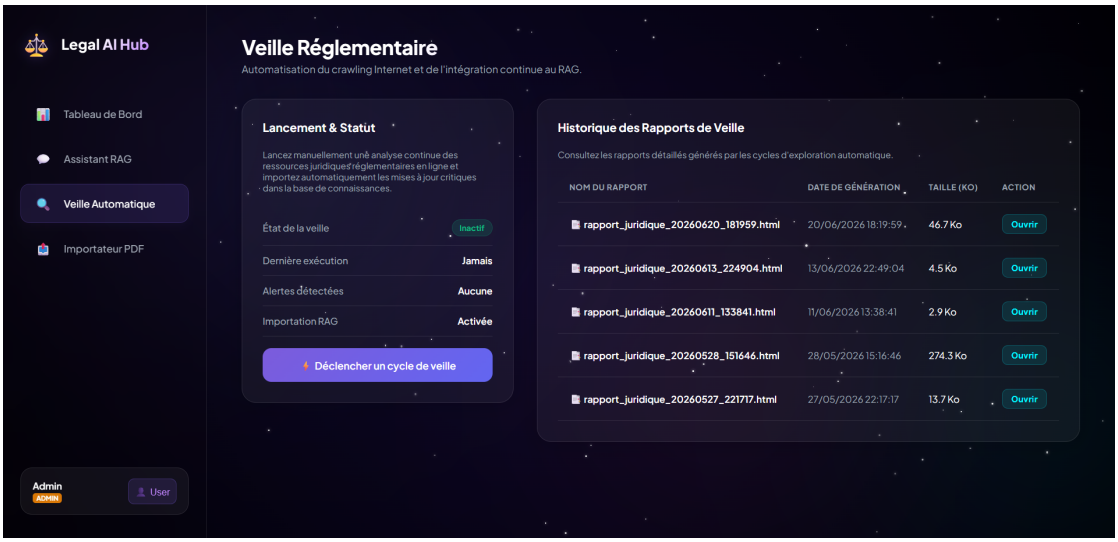


FIGURE A.3 – Module de veille juridique automatique

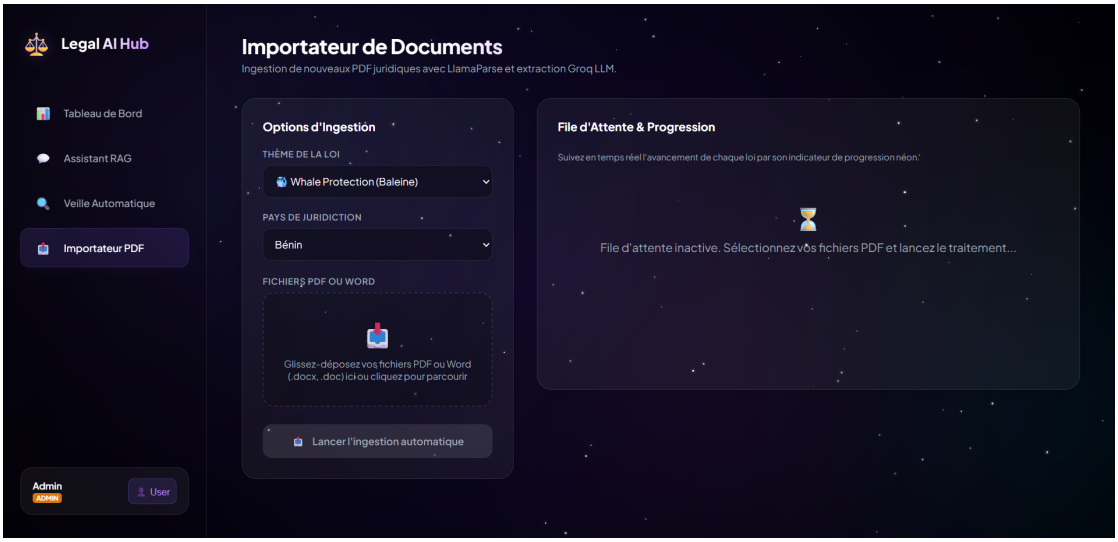


FIGURE A.4 – Pipeline d'ingestion automatique des documents juridiques

Bibliographie

[1] M. Reuter *et al.*, "Towards reliable retrieval in RAG systems for large legal datasets," *arXiv preprint arXiv :2510.06999*, 2025.

[2] H. D. Martim, "Graph RAG for legal norms : A hierarchical and temporal approach," *arXiv preprint arXiv :2505.00039*, 2025.

[3] F. Ariai, J. Mackenzie, and G. Demartini, "Natural language processing for the legal domain : A survey of tasks, datasets, models, and challenges," *arXiv preprint arXiv :2410.21306*, 2025.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[5] Y. Gao, Y. Xiong, X. Gao, K. Lin, X. Fan, R. Diao, J. Wang, X. Pan, R. Wang, and J.-Y. Nie, "Retrieval-augmented generation for large language models : A survey," *arXiv preprint arXiv :2312.10997*, 2023.

[6] C. Sharma, "Retrieval-augmented generation : A comprehensive survey of architectures, enhancements, and robustness frontiers," *arXiv preprint arXiv :2506.00054*, 2025.

[7] S. Gupta, R. Ranjan, and S. N. Singh, "A comprehensive survey of retrieval-augmented generation (RAG) : Evolution, current landscape and future directions," *arXiv preprint arXiv :2410.12837*, 2024.

[8] Y. Gao, Y. Xiong, X. Fan, K. Lin, X. Pan, R. Diao, J. Wang, R. Wang, and J.-Y. Nie, "Modular RAG : Framework for enhanced retrieval-augmented generation," *arXiv preprint arXiv :2407.21059*, 2024.

[9] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG : Learning to retrieve, generate, and critique through self-reflection," *arXiv preprint arXiv :2310.11511*, 2023.

[10] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR : Recursive abstractive processing for tree-organized retrieval," *arXiv preprint arXiv :2401.18059*, 2024.

[11] I. Chalkidis *et al.*, *Natural Language Processing for Legal Texts*. Cambridge University Press, 2022.

[12] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson, "From local to global : A graph RAG approach to query-focused summarization," *arXiv preprint arXiv :2404.16130*, 2024.

[13] L. Martin *et al.*, "CamemBERT : a tasty French language model," *arXiv preprint arXiv :1911.03894*, 2020.

[14] H. Le *et al.*, "FlauBERT : Unsupervised language model pre-training for French," *arXiv preprint arXiv :1912.05372*, 2020.

[15] S. Douka *et al.*, "JuriBERT : A specialized language model for French legal text," *arXiv preprint arXiv :2110.01485*, 2021.

[16] L. Janin *et al.*, "CamemBERTavocat : A legal French language model," in *Proceedings of JURIX 2023*, 2023.

[17] J. Chen *et al.*, "BGE M3-Embedding : Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv :2402.03216*, 2024.

[18] A. Q. Jiang *et al.*, "Mistral 7b," *arXiv preprint arXiv :2310.06825*, 2023.

[19] P. Colombo *et al.*, "SAUL : Legal AI with supervised alignment and unified legal data," *arXiv preprint arXiv :2407.12032*, 2024.

[20] S. Es, J. James, L. E. Anke, and S. Schockaert, "RAGAS : Automated evaluation of retrieval augmented generation," *arXiv preprint arXiv :2309.15217*, 2023.

[21] B. Zhang *et al.*, "Parsebench : Evaluating document parsing quality for retrieval-augmented generation," *arXiv preprint*, 2026.

[22] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.